Web Denetim Merkezi

Kullanım Kılavuzu

Web sitelerini baştan sona test eden ve denetleyen masaüstü programı: site keşfi, kanıtlı tarayıcı testi, belge/link/görsel denetim, yapay zekâ destekli görsel denetim, bulguların canlı doğrulanması, otomatik taşma düzeltme önerisi ve OİBS Bologna aktarımı.

Sürüm 1.4 · Ekim 2026
github.com/mehmetfatihvar/web-denetim-merkezi
Mersin Üniversitesi staj çalışması

İçindekiler

1. Giriş **4**
1.1 Program ne yapar? 4
1.2 Bu kılavuz nasıl okunur? 4
1.3 Temel kavramlar 4

2. Kurulum **6**
2.1 Gereksinimler 6
2.2 Windows'ta kurulum 6
2.3 Linux ve macOS'ta kurulum 6
2.4 İlk açılışta ortam kontrolü 7
2.5 Güncelleme 7

3. Arayüz **9**
3.1 Genel görünüm 9
3.2 Sol menü 9
3.3 Durum şeridi: program çalışıyor mu? 9
3.4 Ana sayfa 11
3.5 Adım sayfaları 11
3.6 Ayrıntılar paneli: ilerleme, kayıt, hatalar, son çalışma 12
3.7 Tema ve kısayollar 15

4. Site profilleri **17**
4.1 Hazır ÖİDB profili 17
4.2 Yeni site eklemek 17
4.3 Profil ayarlarını siteye göre düzenlemek 17
4.4 Profil dosyaları 18

5. Önerilen iş akışı **19**
5.1 Yol haritası 19
5.2 Arka arkaya çalıştırmak 19
5.3 Uzun işler: durdurma ve kaldığı yerden devam 20
5.4 Örnek senaryolar 20

6. Adımlar **22**
6.1 Hazırlık 22
6.2 Web testi 22
6.3 Denetimler 23
6.4 Doğrulama 23
6.5 Yapay zekâ denetimi 23
6.6 Doğrula ve düzelt 25
6.7 Bologna 26
6.8 Teslim 27

7. Sonuçları okumak **28**
7.1 Kanıtlı test raporu (report.html) 28
7.2 Belge ve bağlantı denetimi 28
7.3 Görsel/UI kusur raporu 30
7.4 Kırık sayfa doğrulaması 31
7.5 Yapay zekâ görsel denetimi 32

7.6 Canlı doğrulama (7. aşama) ... 32
7.7 Elle etiketleme (8. aşama) ... 32
7.8 CSS düzeltmesi (9. ve 10. aşamalar) ... 33
7.9 Raporlar ve özet sayfası ... 33
7.10 Adım raporları: her adımın her çalışması ... 34
7.10 Teslim paketi ... 35

8. Yapay zekâ ayarları ... 36
8.1 Sağlayıcılar ve model yazımı ... 36
8.2 Varsayılan modeller ... 37
8.3 Anahtarlar ve güvenlik ... 37
8.4 Kota, hız ve maliyet ... 37
8.5 Modele giden talimat ... 37

9. Ayarlar başvurusu ... 38
9.1 Site profili ... 38
9.2 Yapay zekâ, Bologna, genel ... 40

10. Komut satırı ... 43

11. Klasörler ve dosyalar ... 44

12. Sorun giderme ... 45

13. Sık sorulan sorular ... 47

Ek A. Bologna ... 48
Başka üniversiteler ... 48

Ek B. Geliştirme notları ... 49

1. Giriş

1.1 Program ne yapar?

Web Denetim Merkezi, bir web sitesini baştan sona test eden ve denetleyen bir masaüstü programıdır. Sitenin tıklanabilir bütün sayfalarını gerçek bir tarayıcıda açar, her sayfa için ekran görüntüsü kanıtı alır, bulguları raporlar ve bulguların gerçek olup olmadığını ayrıca doğrular. Bütün işler tek pencereden, sırayla ya da tek tek çalıştırılır; uzun işler durdurulup kaldığı yerden sürdürülebilir.

Program şu soruları cevaplar:

- Sitede hangi sayfalar var? (**site keşfi**)
- Sayfalar açılıyor mu, hangi sayfalar kırık (4xx/5xx)? Güvenlik başlıkları, Türkçe karakter, erişilebilirlik (alt metin, H1, form etiketi), mobil uyum ve JavaScript hataları nasıl? (**kanıtlı test**)
- Belgeler (PDF, Word, Excel) erişilebilir mi, sızdırılmış yerel dosya yolu var mı? Hangi bağlantılar kırık? (**belge ve link denetimi**)
- Sayfalarda taşma, kırık görsel, kesik metin, boş render gibi görünüm hataları var mı? (**görsel/UI denetimi** ve **yapay zekâ destekli görsel denetim**)
- Bulunan hatalar gerçekten var mı, yoksa yanlış alarm mı? (**doğrulama**, **canlı doğrulama**, **elle etiketleme**)
- Taşma hataları hangi CSS ile düzelir, düzeltme başka bir yeri bozar mı? (**CSS denemesi**, **otomatik taşma düzeltme**)
- OİBS Bologna Bilgi Paketi verisi veritabanına nasıl aktarılır? (**Bologna**)

Program siteye özgü değildir: **site profilleri** sayesinde her siteye uygulanır. Mersin Üniversitesi ÖİDB sitesi hazır profil olarak gelir. Yapay zekâ sağlayıcıları ve modeller ayarlardan seçilir.

> **Dikkat**
>
> Program siteyi değiştirmez. Testler yalnızca sayfaları okur; CSS denemeleri yalnızca programın açtığı tarayıcı sekmesine uygulanır. Yine de sunucuya istek gönderildiği için eşzamanlılık ve hız ayarlarını sitenin kaldırabileceği düzeyde tutun (önerilen: 4 eşzamanlı sekme).

1.2 Bu kılavuz nasıl okunur?

- Programı ilk kez kuracaksanız: **2. Kurulum** ve **3. Arayüz**.
- Bir siteyi denetleyecekseniz: **4. Site profilleri** ve **5. Önerilen iş akışı**.
- Bir adımın ne yaptığını ya da hangi dosyayı ürettiğini arıyorsanız: **6. Adımlar**.
- Bir raporun ya da sonucun nasıl okunacağını arıyorsanız: **7. Sonuçları okumak**.
- Bir şey ters gittiyse: **12. Sorun giderme**.

1.3 Temel kavramlar

Kavram	Anlamı
Site profili	Bir sitenin bütün ayarları: adres, ad, keşif kuralları, test parametreleri, yapay zekâ talimatındaki site tanımı. Her site için ayrı profil olur; `profiller/<ad>.json` dosyasında durur.
Site haritası	Test edilecek sayfaların listesi (JSON, `url_list`). Site keşfi üretir; ÖİDB için hazır gelir.

Kavram	Anlamı
Adım	Programın çalıştırdığı tek bir iş (ör. "Tüm site kanıtlı testi"). Her adım bir ya da birkaç aracı çalıştırır ve dosya üretir.
Sıradaki adım	Yol haritasında henüz yapılmamış ilk aşama. Ana sayfadaki **Başlat** onu çalıştırır; **Kalanları da sırayla çalıştır** sonraki aşamaları da arka arkaya çalıştırır.
Şablon / temsilci	Aynı düzende üretilen sayfalar (ör. bütün haber detayları) bir şablondur. Görsel hatalar şablona bağlı olduğundan her şablondan birkaç temsilci sayfa denetlenir.
Kanıt	Her sayfanın masaüstü (1366 px) ve mobil (375 px) tam sayfa ekran görüntüsü.
Çıktı klasörü	Bir profilin bütün çıktılarının yazıldığı klasör: `ciktilar/<profil>/`.
Teslim paketi	Bütün rapor ve verilerin anlaşılır adlarla toplandığı `TESLIM/` klasörü ve içindekiler sayfası.

2. Kurulum

2.1 Gereksinimler

Yazılım	Gerekli mi?	Ne için
Python 3.9 veya üstü	Evet	Programın ve bütün araçların çalışması. python.org'dan kurulan Python, arayüz için gereken Tkinter'ı da içerir.
İnternet bağlantısı	Kurulumda evet	Python paketleri ve Chromium tarayıcısı indirilir (yaklaşık 300 MB).
Disk alanı	Evet	Program ve paketler ~1 GB. Ekran görüntüleri site büyüklüğüne bağlıdır: ÖİDB'nin tamamı (17.394 sayfa) için ~8–10 GB.
Docker Desktop	Yalnız Bologna için	Bologna'nın PostgreSQL veritabanı ve tarayıcısı Docker ile çalışır.
Go 1.22+ ve PostgreSQL	Yalnız Bologna için, Docker yerine	Bologna'yı Docker olmadan çalıştırmak için.
Yapay zekâ API anahtarı	Yalnız yapay zekâ adımları için	Ör. Google Gemini (ücretsiz katman yeterli). Yerel Ollama ile anahtarsız da çalışır.
Ollama	İsteğe bağlı	Yapay zekâ incelemesini kendi bilgisayarınızda, ücretsiz ve verisiz-dışarı çalıştırmak için.

2.2 Windows'ta kurulum

1. Python'u kurun: https://www.python.org/downloads/ adresinden Python 3.9 veya üstünü indirin. Kurulumun ilk ekranında **"Add python.exe to PATH"** kutusunu işaretleyin.
2. Programı indirin: `git clone https://github.com/mehmetfatihvar/web-denetim-merkezi.git` ya da GitHub'da **Code > Download ZIP** ile indirip bir klasöre açın.
3. Klasördeki **BASLAT.bat** dosyasına çift tıklayın. İlk açılışta **KURULUM.bat** kendiliğinden çalışır: Python'u bulur, sürümünü denetler, klasörün içinde ayrı bir sanal ortam (`.venv`) kurar, paketleri ve Chromium tarayıcısını indirir. Birkaç dakika sürer.
4. Kurulum bitince program açılır. Sonraki açılışlarda yalnızca **BASLAT.bat** yeterlidir.

> **İpucu**
>
> Program bilgisayardaki diğer Python kurulumlarına dokunmaz; her şey klasörün içindeki `.venv` sanal ortamına kurulur. Programı kaldırmak için klasörü silmek yeterlidir.

> **Dikkat**
>
> Klasörü OneDrive, Dropbox ya da Masaüstü gibi eşitlenen bir yere koymayın: binlerce ekran görüntüsünün eşitlenmesi hem yavaşlatır hem kotayı doldurur.

2.3 Linux ve macOS'ta kurulum

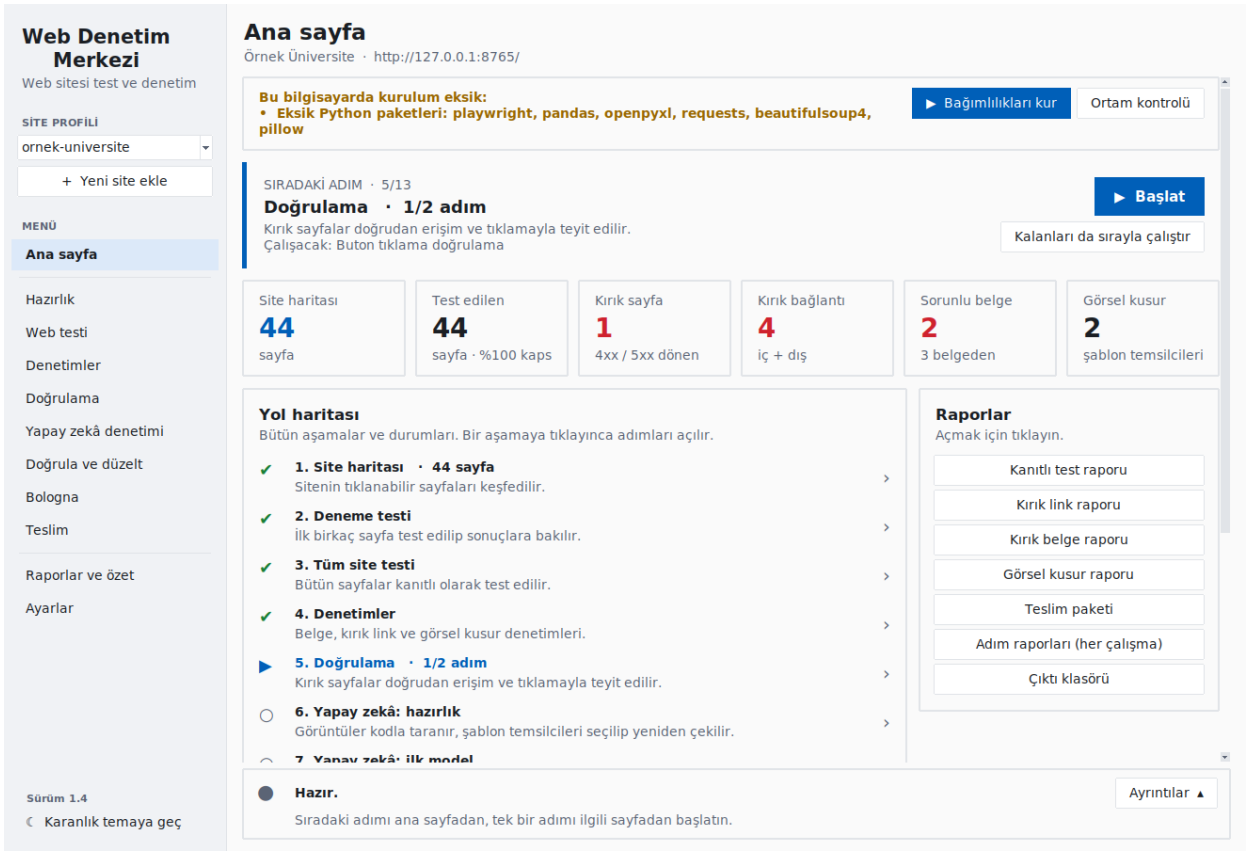
```
git clone https://github.com/mehmetfatihvar/web-denetim-merkezi.git
cd web-denetim-merkezi
```

```
./baslat.sh          # ilk açılışta ./kurulum.sh kendiliğinden çalışır
```

Debian/Ubuntu'da arayüz ve sanal ortam için şu paketler gerekebilir: `sudo apt install python3-tk python3-venv`. macOS'ta Homebrew Python kullanıyorsanız: `brew install python-tk`.

2.4 İlk açılışta ortam kontrolü

Program her açılışta, araçları çalıştıracak Python'da gerekli paketlerin ve Chromium tarayıcısının kurulu olup olmadığını arka planda denetler. Eksik varsa ana sayfanın en üstünde bir uyarı kartı çıkar:



Şekil 1. *Kurulum eksikse ana sayfada çıkan uyarı ve "Bağımlılıkları kur" düğmesi*

Bağımlılıkları kur düğmesi eksikleri kurar. Ayrıntılı döküm için **Hazırlık > Ortam kontrolü** adımını çalıştırın: her paket, Chromium, Go ve Docker için kurulu/eksik bilgisi çalışma kaydına yazılır.

2.5 Güncelleme

1. Programı kapatın. Çalışan bir adım varsa önce **Durdur** ile durdurun.
2. **ZIP ile:** GitHub'da depo sayfasında **Code > Download ZIP** ile projeyi indirin. ZIP'teki `web-denetim-merkezi-main` klasörünün **içindekilerin hepsini** program klasörünün üzerine kopyalayın ve "Hedefteki dosyaları değiştir" deyin.
3. **git ile:** program klasöründe `git pull` çalıştırın.
4. **KURULUM.bat**'ı (Linux/macOS: `./kurulum.sh`) bir kez çalıştırın; yeni paketler varsa kurulur.
5. Programı açın. Yarıda kalan bir adım varsa aynı adımı yeniden çalıştırın; kaldığı yerden devam eder.

`ciktilar/` (bütün sonuçlar), `ayarlar.json` (anahtarlar dahil), `.venv/` (kurulum) ve kendi oluşturduğunuz profiller ZIP'te yoktur; güncellemede üzerlerine yazılmaz.

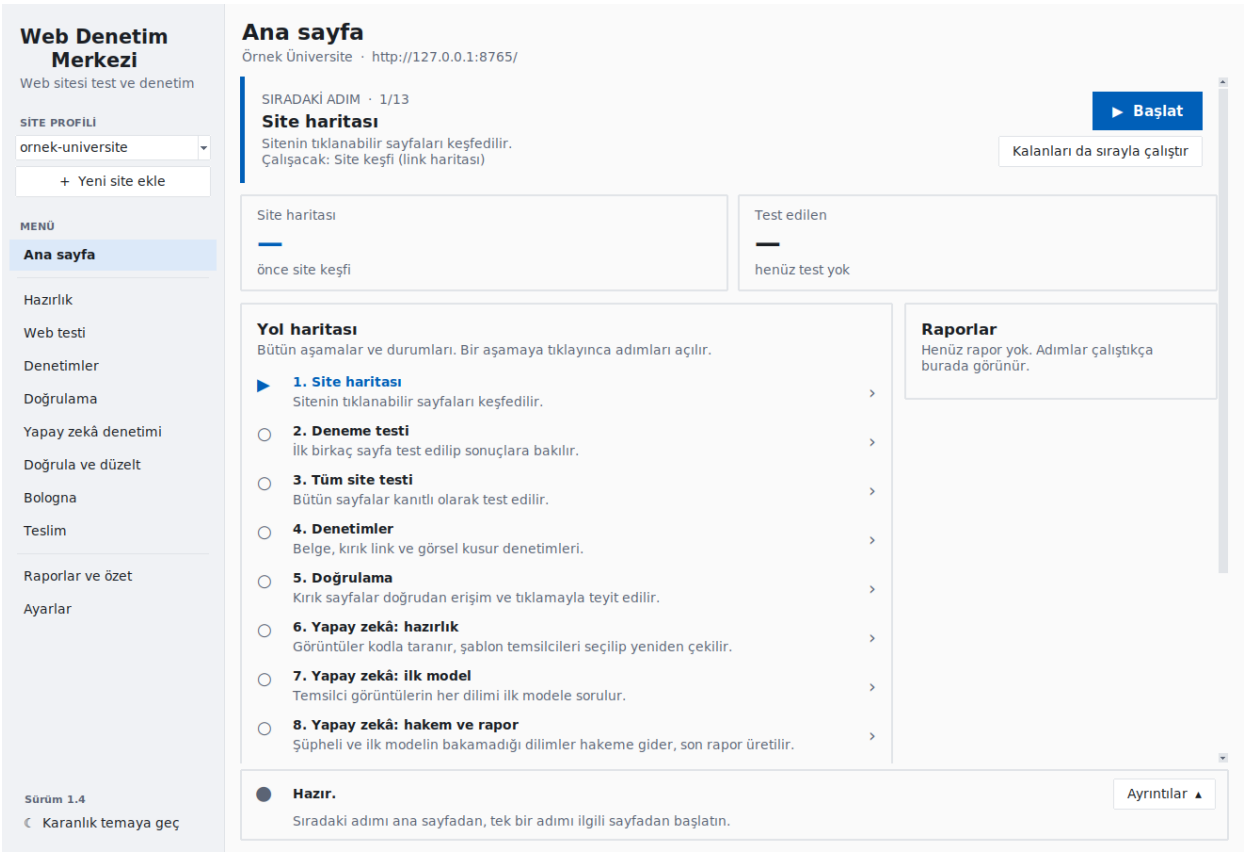
> **Dikkat**
>
> Dosyaları GitHub'dan **tek tek** indirerek güncellemeyin. Program dosyaları birbirine bağlıdır; yeni bir dosya, sizde olmayan başka bir yeni dosyaya ihtiyaç duyabilir ve program açılmaz. Ayrıca dosya sayfasında "Farklı kaydet" Python kodu yerine sayfanın HTML'ini kaydeder. Her zaman yukarıdaki ZIP ya da git yolunu kullanın.

3. Arayüz

3.1 Genel görünüm

Pencere üç bölümden oluşur: solda **menü**, sağda seçilen **sayfa**, en altta her zaman görünen **durum şeridi**. Çalışma kaydı, hatalar ve son çalışma sekmeleri şeritteki **Ayrıntılar** düğmesiyle açılan panelde durur; açıkken sayfa ile panel arasındaki çizgi sürüklenerek alanlar büyütülüp küçültülebilir.

Çalıştırmanın üç yolu vardır: ana sayfadaki **Başlat** (sıradaki aşama), adım sayfalarındaki **Çalıştır** (tek adım) ve adım sayfalarındaki **Bu sayfadakileri sırayla çalıştır…** (o sayfanın adımları).



Şekil 2. *Ana sayfa: Sıradaki adım kartı ve tek Başlat düğmesi; altta durum şeridi*

3.2 Sol menü

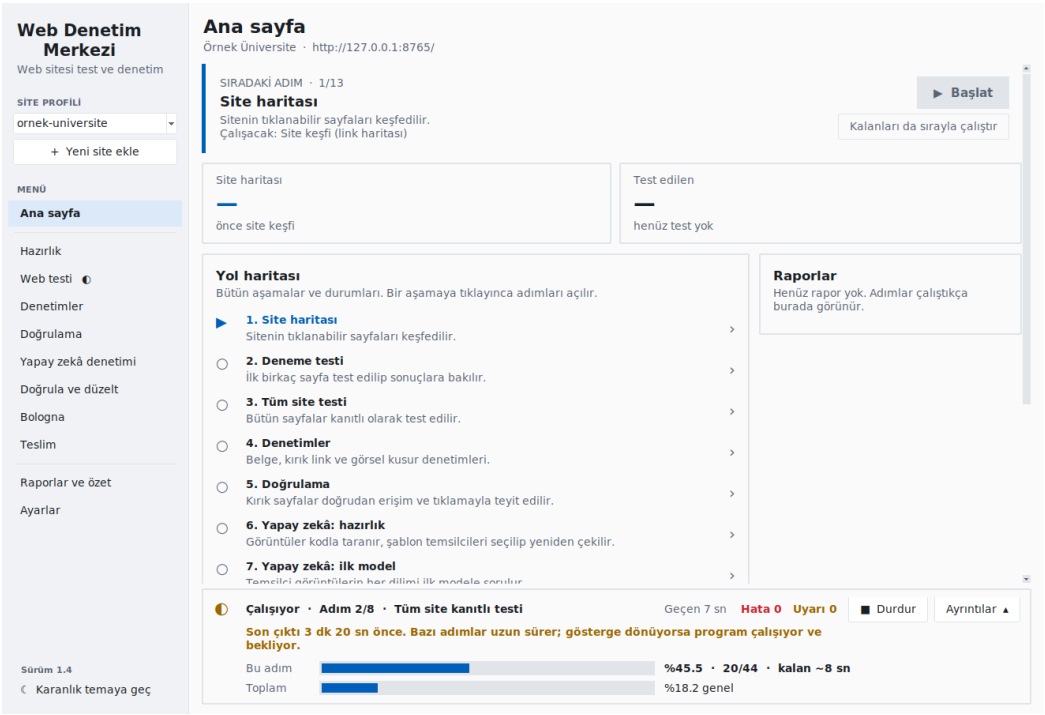
- **Site profili:** hangi sitenin denetleneceği. Açılır listeden seçilir; **+ Yeni site ekle** yeni profil oluşturur (bkz. 4.2).
- **Sayfalar:** Ana sayfa; adım sayfaları (Hazırlık, Web testi, Denetimler, Doğrulama, Yapay zekâ denetimi, Doğrula ve düzelt, Bologna, Teslim); Raporlar ve özet; Ayarlar.
- **Tema düğmesi** (en altta): açık ve karanlık tema arasında geçiş. Seçim hatırlanır.

3.3 Durum şeridi: program çalışıyor mu?

Pencerenin en altındaki şerit, hangi sayfada olursanız olun programın ne yaptığını gösterir:

Öğe	Ne gösterir
Dönen gösterge (◐ ◓ ◑ ◒)	Bir adım çalışırken saniyede dört kez döner. Dönüyorsa program açık ve çalışıyordur. Biten çalışmada ✔ (başarılı), ✖ (sorun var) ya da ■ (durduruldu) olur.

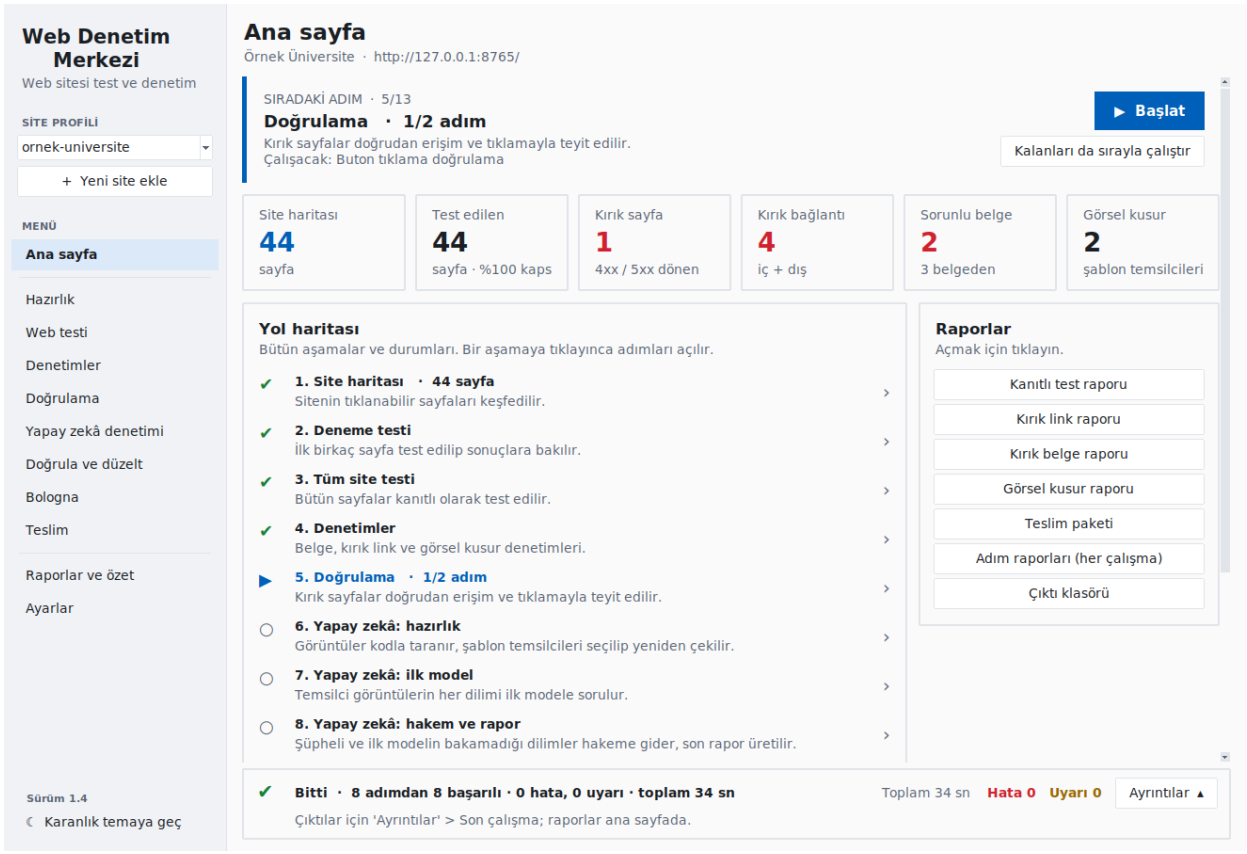
Öğe	Ne gösterir
Çalışıyor · Adım 2/8 · ad	Çalışan adım ve kaçıncı adım olduğu. Sol menüde o adımın sayfasının yanında da küçük bir dönen gösterge görünür.
Son hareket satırı	Aracın en son ne zaman çıktı verdiği: "araç az önce çıktı verdi", "son hareket 40 sn önce". **2 dakika** çıktı gelmezse satır sarıya döner ve bunun normal olabileceğini söyler (yapay zekâ, büyük sayfalar). **10 dakika** çıktı gelmezse adımın takılmış olabileceğini ve ne yapılacağını yazar.
Geçen süre, Hata, Uyarı	Çalışmanın süresi ve sayaçlar. Sayaçlara tıklamak hata listesini açar.
■ Durdur	Yalnız çalışırken görünür. Araç önce kısmi sonucunu kaydeder; sonra tarayıcılar dahil bütün alt süreçler kapatılır. Sıradaki adımlar çalışmaz.
Ayrıntılar ▲	Çalışma kaydı, hatalar ve son çalışma panelini açar/kapatır. Bir adım başarısız olursa panel kendiliğinden açılır.
Bu adım / Toplam	Çalışırken iki ilerleme çubuğu: adımın yüzdesi, işlenen/toplam ve kalan süre; bütün çalışmanın yüzdesi.



Şekil 3. *Araç 2 dakikadan uzun süredir çıktı vermediğinde şerit: gösterge dönmeye devam eder, satır sarıya döner*

Hata olunca sonraki adımlarla sürdürmek isterseniz **Ayarlar > Genel > Hata olursa sonraki adımlarla sürdür**'ü açın. Kapalıyken (varsayılan) bir adım başarısız olursa sıradakiler çalışmaz.

3.4 Ana sayfa

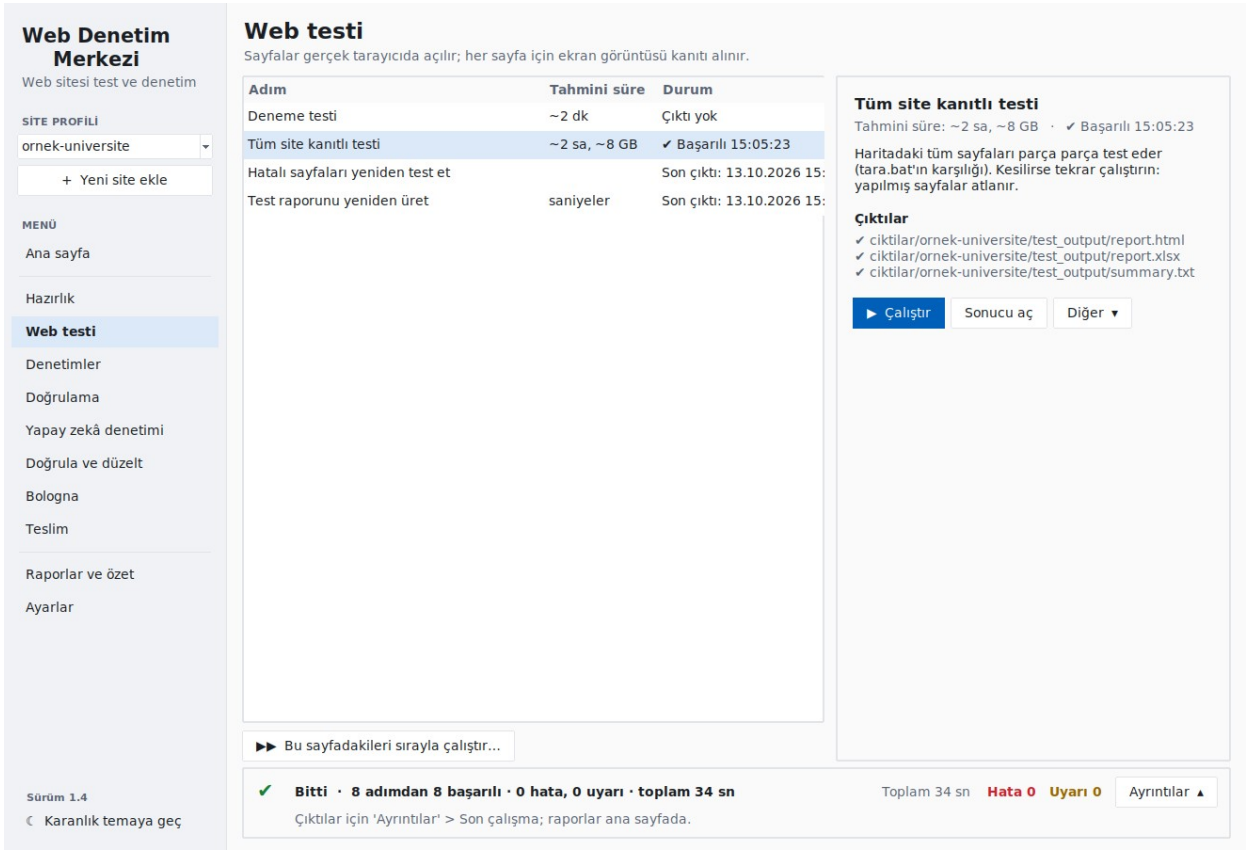


Şekil 4. *Ana sayfa: özet kutucukları, yol haritası ve rapor kısayolları*

- **Özet kutucukları:** site haritasındaki sayfa sayısı, test edilen sayfa ve kapsam, kırık sayfa (4xx/5xx), kırık bağlantı, görsel kusur, kesin yapay zekâ bulgusu. Yalnızca üretilmiş çıktılar için kutucuk görünür.
- **Sıradaki adım kartı:** en üstte, yol haritasında henüz yapılmamış ilk aşama ve ne yaptığı yazar. ▶ **Başlat** onu çalıştırır (bir aşama yarım kalmışsa, ör. "2/4 adım", yalnız eksik adımlar çalışır). **Kalanları da sırayla çalıştır** sonraki aşamaları da arka arkaya çalıştırır; başlamadan listeyi ve sizin yapmanız gerekenleri gösterip onay ister. Etiketlemede durmaz: etiketleme sayfasını (8a) oluşturur, sizi bekleyen 8b'yi atlayıp taşma düzeltme ve teslimle devam eder; sonunda etiketleme sayfasını açmayı önerir.
- **Yol haritası:** önerilen iş sırası. Tamamlanan aşamalar ✓, sıradaki ▶ ile işaretlidir. Bir satıra tıklamak o aşamanın adım sayfasını açar.
- **Raporlar:** yalnız üretilmiş raporların kısayolları görünür.

3.5 Adım sayfaları

Her adım sayfasında solda adımların **listesi**, sağda seçili adımın **ayrıntı kartı** vardır.



Şekil 5. *Web testi sayfası: adım listesi ve seçili adımın ayrıntı kartı*

- **Liste sütunları:** adım adı, tahmini süre, durum. Durum; son çıktının zamanını, çalışırken canlı yüzdeyi (● %45), bitince sonucu (✓ Başarılı / ✗ Başarısız) gösterir.
- **Çalıştırmak:** listeden adımı seçip kartın ▶ **Çalıştır** düğmesine basın. Bir çalışma sürerken düğme pasiftir.
- **Sayfadakilerin hepsi:** listenin altındaki ▶▶ **Bu sayfadakileri sırayla çalıştır…** sayfadaki adımları bir listede gösterir; hepsi işaretli gelir. Yalnız aynı işi iki kez yapan, ön koşul isteyen ya da sizi bekleyen birkaç adım işaretsizdir ve nedeni yanında yazar (ör. Bologna'da "Seviye tara · Tüm siteyi tara kapsar", "Birim testleri · Go kurulu olmalı"; Doğrula ve düzelt'te 8b). İşaretleri değiştirip **Başlat**'a basın; seçilenler listedeki sırayla çalışır.
- **Ayrıntı kartı:** adımın açıklaması, ürettiği dosyalar (✓ var / ○ yok) ve üç düğme: ▶ **Çalıştır**, **Sonucu aç** (adımın ana çıktısı) ve **Diğer ▾**. Diğer menüsünde **Son adım raporu** (bkz. 7.10), **Klasörü aç** ve **Çalışacak komutu göster** (adımı çalıştırmadan hangi komutun çalışacağını kayda yazar) bulunur.

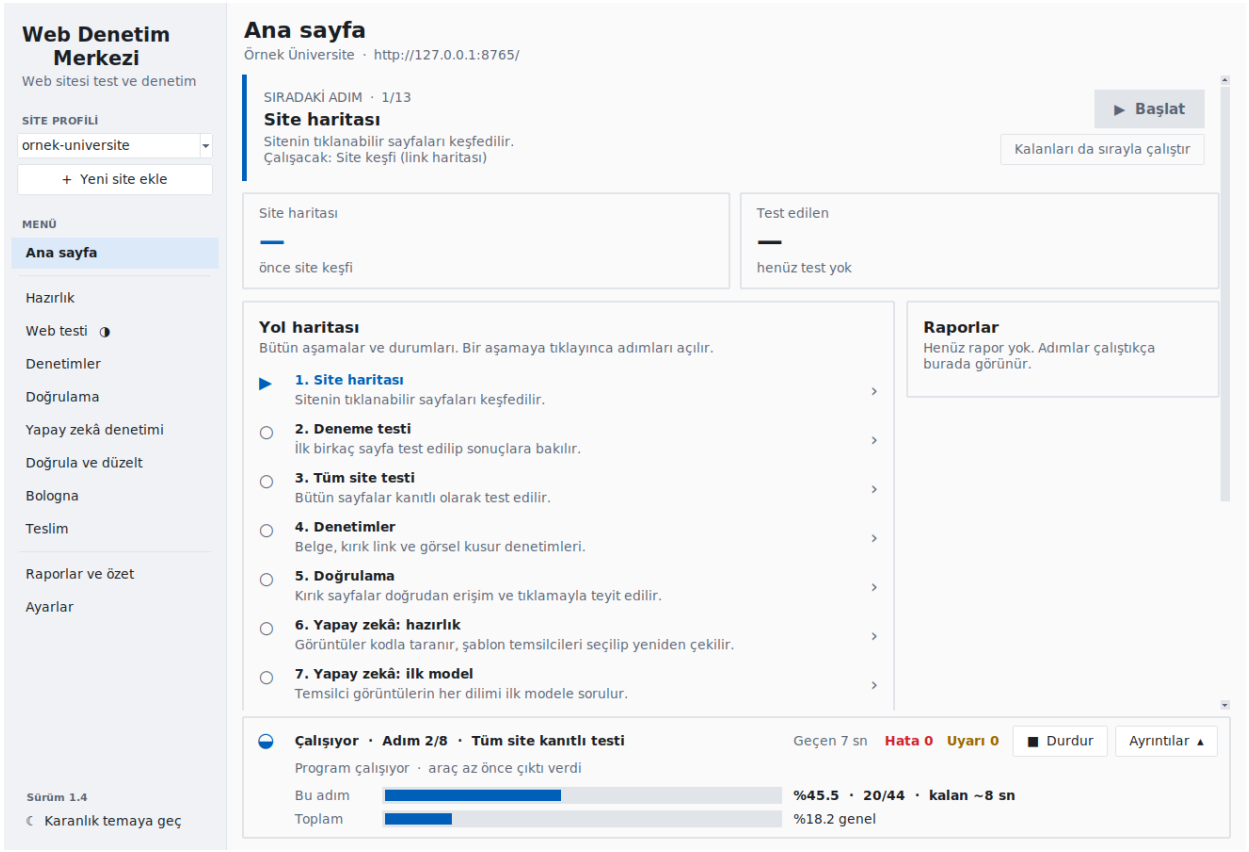
3.6 Ayrıntılar paneli: ilerleme, kayıt, hatalar, son çalışma

İlerleme

Bir çalışma başlayınca durum şeridinde iki ilerleme çubuğu görünür:

- **Bu adım:** çalışan adımın yüzdesi, işlenen/toplam sayısı (ör. 946/2.000), parçalı testte kaçıncı parça olduğu ve **kalan süre tahmini**. Tahmin son 10 dakikadaki hıza göre yapılır, sunucu yavaşlar ya da hızlanırsa kendini günceller. Sayfalar eşzamanlı test edildiği için sayı, listedeki sıra değil bitirilen sayfa sayısıdır; tam testte zaten yapılmış olduğu için atlanan parçalar hız hesabına katılmaz.
- **Toplam:** seçilen bütün adımların genel yüzdesi.

- Sağda **geçen süre**, kırmızı **Hata** ve sarı **Uyarı** sayaçları. Sayaçlara tıklamak hata listesini açar. Pencere başlığında da genel yüzde görünür.



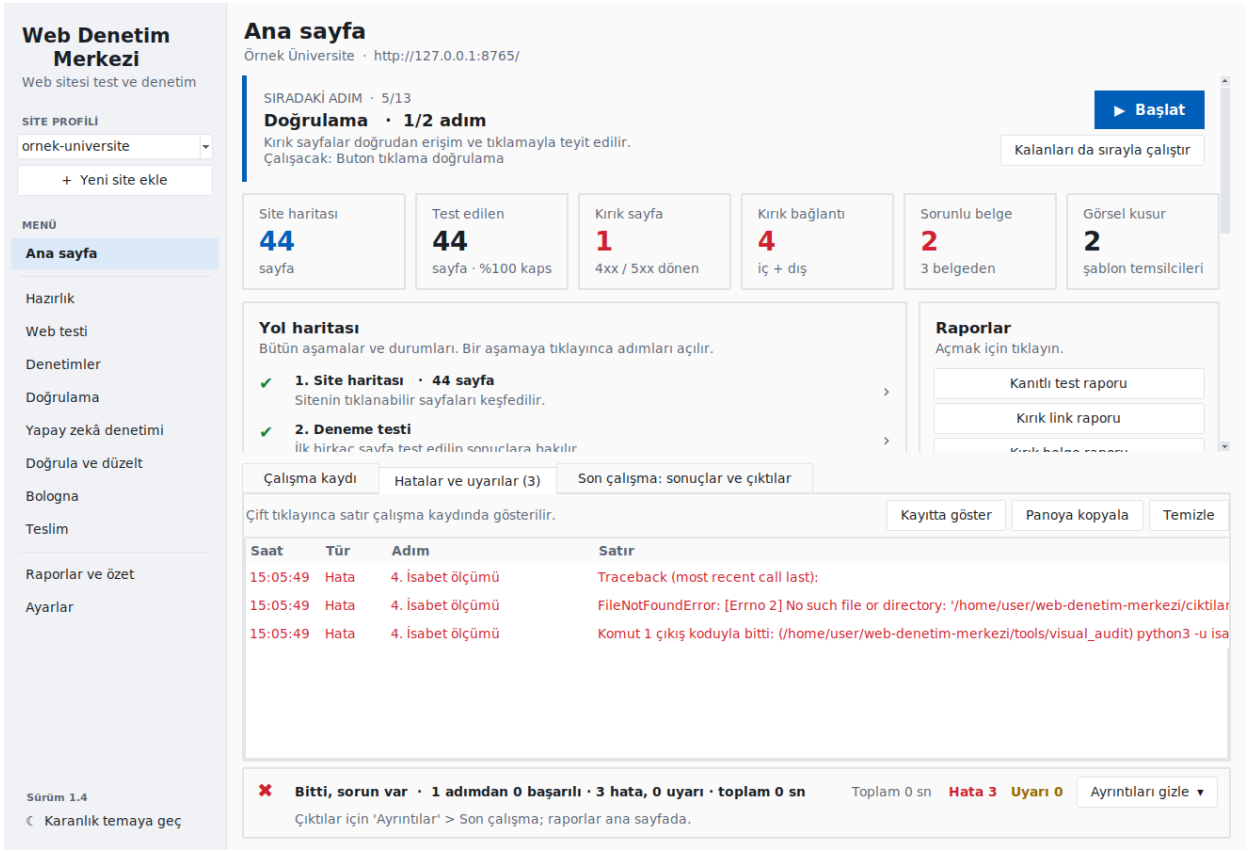
Şekil 6. *Tüm site testi çalışırken: dönen gösterge, son hareket, adım ve toplam ilerlemesi, kalan süre*

Çalışma kaydı

Araçların canlı çıktısıdır. Hata satırları kırmızı, uyarılar sarı görünür. **Kayıtta ara** ile metin aranır. **Yeni satırları takip et** kapatılırsa eski satırlar kaymadan okunur. Her adımın tam kaydı ayrıca `ciktilar/<profil>/merkez_kayitlari/` klasörüne tarih ve adım adıyla yazılır (**Kayıt klasörü** düğmesi).

Hatalar ve uyarılar

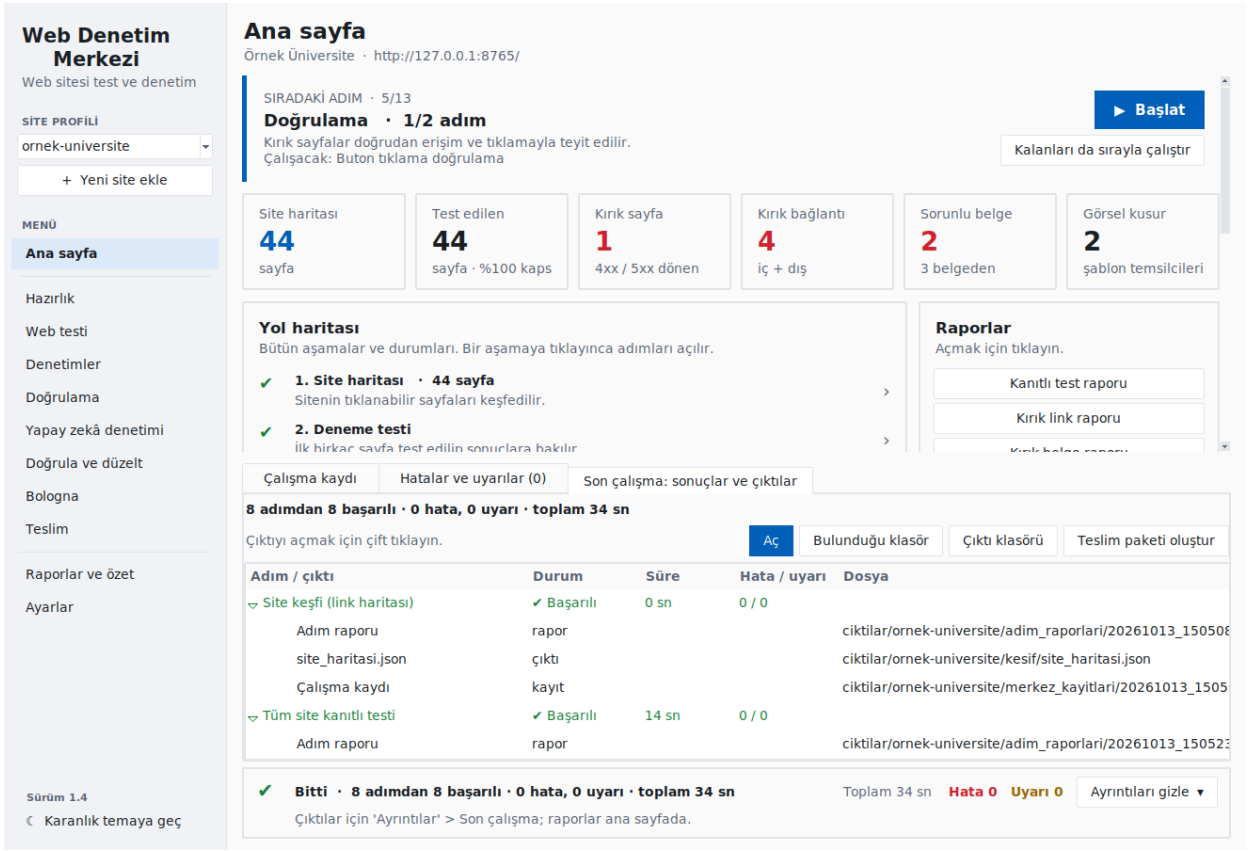
Araç çıktısında yakalanan hatalar (Python hata dökümü, `[HATA]` satırları, sıfır olmayan çıkış kodu, bulunamayan program) ve uyarılar (`[!]` satırları, açılamayan ya da zaman aşımına uğrayan sayfa) saat ve adım adıyla listelenir. Satıra **çift tıklamak** çalışma kaydında o satıra gider ve vurgular. **Panoya kopyala** listeyi destek istemek için kopyalar.



Şekil 7. *Hatalar ve uyarılar sekmesi*

Son çalışma

Ayrıntılar > Son çalışma sekmesindedir: her adımın durumu, süresi, hata/uyarı sayısı, **ürettiği çıktı ve raporlar** ve kayıt dosyası. Bir satıra çift tıklamak dosyayı açar; **Bulunduğu klasör** klasörü açar; **Teslim paketi oluştur** paketi hazırlar.

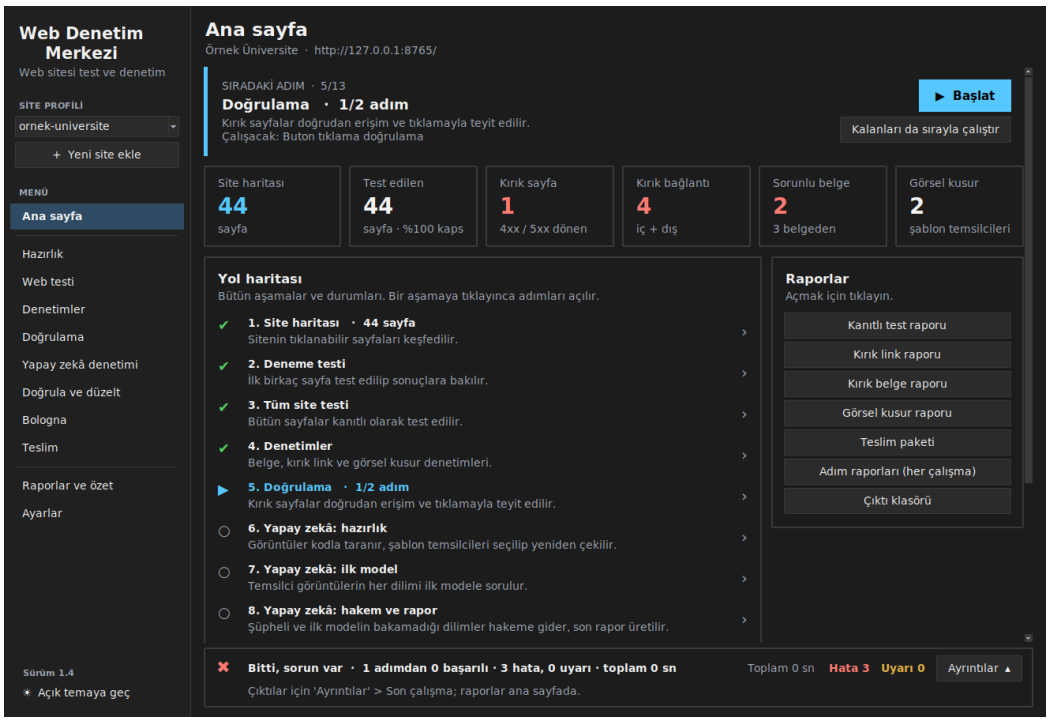


Şekil 8. *Son çalışma sekmesi: adımlar, süreler ve üretilen dosyalar*

Her adımın altındaki ilk satır **Adım raporu**dur: o adımın bu çalışmadaki sonucunu tek sayfada gösterir (bkz. 7.10).

3.7 Tema ve kısayollar

Sol alttaki düğme açık ve karanlık tema arasında geçiş yapar. Varsayılan **sade görünüm** hızlı çizilir. İsteyen **Ayarlar → Genel → Arayüz görünümü** ile Windows 11 tarzı **modern** görünümü (`sv-ttk`) seçebilir; ancak bu tema her öğeyi resimle çizdiği için Windows'ta sayfa geçişleri ve tema değişimi belirgin yavaşlar (ölçümde 5–7 kat). Değişiklik program yeniden açılınca geçerli olur.



Şekil 9. *Karanlık tema*

Kısayol	İşlev
F5	Durum ve özetleri yenile
Enter (arama kutusunda)	Kayıtta önceki eşleşmeyi bul

4. Site profilleri

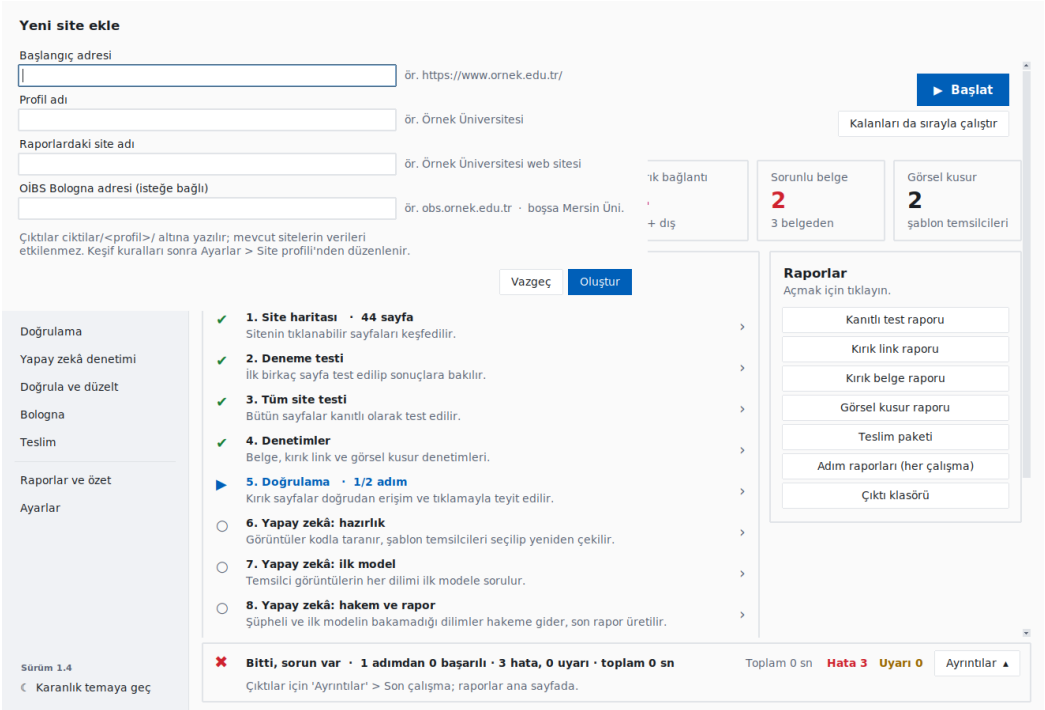
Profil, bir sitenin bütün ayarlarını tutar. Profil değiştirmek o sitenin çıktılarını ve ayarlarını getirir; yapay zekâ anahtarları gibi genel ayarlar bütün profillerde ortaktır.

4.1 Hazır ÖİDB profili

`meu-oidb` profili Mersin Üniversitesi ÖİDB ön-prodüksiyon sitesi (`https://193.255.182.40/`) içindir. 17.394 sayfalık site haritası ve şablon kümeleri `data/meu-oidb/` klasöründe hazır gelir; bu yüzden **keşif adımını atlayıp doğrudan teste geçebilirsiniz**. Site kurum ağındadır; dışarıdan erişim için VPN gerekebilir.

4.2 Yeni site eklemek

1. Sol menüde **+ Yeni site ekle**'ye basın.
2. **Başlangıç adresi** (ör. `https://www.ornek.edu.tr/`), **profil adı** ve raporlarda görünecek **site adı**nı girin. Üniversite OİBS kullanıyorsa **OİBS Bologna adresi**'ni de yazabilirsiniz (ör. `obs.ornek.edu.tr`; boş bırakılırsa Mersin Üniversitesi). **Oluştur**'a basın.
3. Yeni profil seçili olarak açılır. Çıktıları `ciktilar/<profil>/` altında toplanır; diğer sitelerin verileri etkilenmez.
4. Ana sayfadaki **Başlat** ile ilerleyin: önce site keşfi, sonra deneme testi, sonra tüm site testi ve denetimler.



Şekil 10. *Yeni site ekleme penceresi*

4.3 Profil ayarlarını siteye göre düzenlemek

Ayarlar > Site profili sekmesindeki ayarların tam listesi 9. bölümdedir. En çok ihtiyaç duyulanlar:

- **Keşif kuralları:** Site linkleri JavaScript ile üretiyorsa (SPA: React, Vue …) **Tarayıcıyla keşfet**'i açın. Filtre, sıralama, takvim gibi sonsuz kombinasyon üreten adresleri **Hariç desenler**'e yazın (ör. `/arama /etiket/ /takvim`). Gereksiz sorgu parametrelerini (ör. `sort filtre`

`sayfa_boyutu`) **Silinecek sorgu parametreleri**'ne ekleyin. **Yol başına sorgu varyasyonu** ikinci bir güvencedir.

- **Saniyede istek ve eşzamanlılık:** sunucuyu yormamak için düşük tutun. Zaman aşımı hataları artarsa eşzamanlılığı düşürün.
- **Tarayıcı dili ve User-Agent:** testin hangi dilde ve hangi kimlikle yapılacağı.
- **Yapay zekâ talimatı:** sitenin İngilizce tanımı (ör. `an e-commerce website`) ve dili (ör. `German`). Modele verilen talimat bunlarla kurulur.

Değişikliklerden sonra **Kaydet**'e basın. Kaydedilmeyen değişiklikler yine de o oturumdaki çalışmalarda kullanılır; **Değişiklikleri geri al** formu kayıtlı haline döndürür.

4.4 Profil dosyaları

Profiller `profiller/<ad>.json`, genel ayarlar `ayarlar.json` dosyasındadır. Bir profili başka bir bilgisayara taşımak için profil dosyasını kopyalamak yeterlidir; yollar program klasörüne göreli tutulur. `ayarlar.json` ve `ciktilar/` git'e girmez.

5. Önerilen iş akışı

5.1 Yol haritası

Ana sayfadaki yol haritası şu sırayı önerir. Her aşama bir öncekinin çıktısını kullanır:

#	Aşama	Neden bu sırada?
1	Site haritası (keşif)	Test edilecek sayfaların listesi olmadan hiçbir adım çalışmaz.
2	Deneme testi	İlk 50 sayfayla bağlantıyı, raporu ve süreyi görürsünüz. Sorun varsa tüm siteye geçmeden fark edilir.
3	Tüm site testi	Bütün sayfalar test edilir, ekran görüntüleri alınır. Görsel ve yapay zekâ denetimi bu görüntüleri kullanır.
4	Denetimler	Belge, kırık link ve şablon bazlı görsel kusur denetimi.
5	Doğrulama	Kırık görünen sayfalar doğrudan açılarak ve butonlara tıklanarak teyit edilir.
6	Yapay zekâ: hazırlık	Görüntüler kodla taranır, her şablondan temsilciler seçilip yeniden çekilir.
7	Yapay zekâ: ilk model	Temsilcilerin her dilimi ilk modele (Gemma) sorulur. Satırda kapsam görünür (ör. 3990/4084 dilim, 94 yapılamadı). Hiç denenmemiş ya da kota yüzünden kalan dilim varsa aşama tamam sayılmaz.
8	Yapay zekâ: hakem ve rapor	Şüpheli dilimler ve ilk modelin bakamadığı dilimler hakeme gider; isabet, metin kontrolü ve son rapor üretilir. İlk model tamamlanmadan başlamaz.
9	Canlı doğrulama	Bulgular canlı sitede yeniden ölçülür: gerçek mi, yanlış alarm mı? (7. adım)
10	Elle etiketleme: sayfa	Etiketleme sayfası oluşturulur; açıp bulguları 1/2/3 ile işaretleyin, 'CSV indir'e basın (8a).
11	Elle etiketleme: sonuç	İndirilen etiketlerden isabet hesaplanır (8b). Etiketlemeden sonra ayrı çalıştırın.
12	Taşma düzeltme	Taşma için CSS önerisi üretilir ve canlı sayfada denenir (10 ve 9. adımlar).
13	Teslim paketi	Bütün çıktılar tek klasörde toplanır.

5.2 Arka arkaya çalıştırmak

Ana sayfadaki **Kalanları da sırayla çalıştır**, sıradaki aşamadan başlayıp yol haritasının kalanını arka arkaya çalıştırır. Önce çalışacak adımların listesini gösterip onay ister; zincirde etiketleme varsa onay penceresi bunu ⚠️ ile belirtir. Etiketlemenin kendisi sizin işiniz: zincir etiketleme sayfasını (8a) oluşturur ama sizi beklemez, 8b'yi atlayıp devam eder. Bitince program etiketleme sayfasını açmayı önerir; işaretleyip **CSV indir**'e basın, sonra ana sayfada **Başlat** 8b'yi çalıştırır. Teslim paketini etiketlerle birlikte istiyorsanız 8b'den sonra Teslim sayfasından yeniden oluşturun.

Komut satırından aynı zincirler `--calistir tam-hat` ve `--calistir ai-hatti` ile çalıştırılabilir (bkz. 10). **Tam hat** adımları:

1. Tüm site kanıtlı testi
2. Belge denetimi (PDF/Word)
3. Kırık link denetimi
4. Şablon kümeleme
5. Görsel/UI kusur denetimi
6. Kırık sayfa doğrulama (doğrudan erişim)

7. Buton tıklama doğrulama
8. Teslim paketini oluştur

Yapay zekâ hattı adımları:

1. 1. Kodla piksel taraması
2. 2a. Temsilci URL listesi
3. 2b. Kaydırarak yeniden çekim
4. 3b. Yapay zekâ analizi
5. 3c. Hakem modeli
6. 4. İsabet ölçümü
7. 5. Metin kodlama denetimi
8. 6. Görsel denetim son raporu
9. 7. Bulguları canlı sitede doğrula

5.3 Uzun işler: durdurma ve kaldığı yerden devam

- Tüm site testi büyük sitelerde saatler sürer; bilgisayarı uyku moduna almayın, şarja takın.
- **Durdur** ile ya da program kapanınca yarıda kalan adımı **yeniden çalıştırmanız yeterlidir**: test, keşif, yapay zekâ analizi ve Bologna kaldığı yerden devam eder. Yapılmış sayfalar atlanır.
- Tüm site testinde **Tam testi sıfırdan başlat** (Ayarlar) açıksa önceki sonuçlar silinip baştan başlanır; normalde kapalı tutun.
- Ücretsiz Gemini kotası günlük dolarsa yapay zekâ analizi durur; ertesi gün aynı adımı çalıştırın.
- **Hata alan istekler de yeniden denenir.** Yapay zekâ analizi yalnız başarılı cevapları "yapıldı" sayar. Sunucu hatası (ör. Gemma 500), zaman aşımı ya da kota hatası alan istekler aynı çalıştırmada otomatik olarak **2 tur daha** denenir (kayıtta "Yeniden deneme turu 1/2"); adım sonradan tekrar çalıştırılırsa yine yalnız yapılamayanlar gönderilir.
- **İlk model ve hakem ayrı aşamalardır.** 3b bitince kayda kapsam yazılır: "İlk model (gemini:gemma-4-31b-it): 3990/4084 dilim tamam, 94 yapılamadı". Hiç denenmemiş (yarıda kalmış) ya da kota yüzünden yapılamamış dilim varsa 3b **başarısız** biter ("İlk model tamamlanmadı") ve hat hakeme geçmez; 3b'yi tekrar çalıştırın. 3c de başlamadan aynı kontrolü yapar. 3 denemede de yapılamayan dilimler (Gemma 500, zaman aşımı) ise hakeme gider; denetimsiz kalmaz.
- **Program çalışıyor mu?** Durum şeridindeki gösterge dönüyorsa program açıktır; "son hareket" satırı aracın en son ne zaman çıktı verdiğini söyler (bkz. 3.3).
- **Yapay zekâ analizinin çalıştığını nasıl anlarım?** Kayda yalnız hata alan istekler tek tek yazılır; başarılı cevaplar sessizce `sonuclar.jsonl` dosyasına kaydedilir. Her 20 istekte bir `[340/3792] ✓ 270 başarılı ✗ 70 yapılamadı ~906 dk kaldı` gibi bir ilerleme satırı çıkar. Gemma yavaş olduğu için iki satır arası 8–15 dakika sürebilir. Cevap vermeyen bir bağlantı 5 dakikada zaman aşımına uğrar ve yeniden denenir, yani istek asılı kalmaz. 30 dakikadan uzun süre yeni satır gelmezse (ör. bilgisayar uykuya geçtiyse) adımı durdurup yeniden başlatın; tamamlananlar atlanır.

5.4 Örnek senaryolar

ÖİDB sitesinin tamamını denetlemek

1. Profil: `meu-oidb`. Gerekirse VPN'e bağlanın.
2. **Web testi > Deneme testi** ile bağlantıyı kontrol edin.

3. Ana sayfada **Kalanları da sırayla çalıştır**'a basın (harita hazır olduğu için keşif atlanır). Yapay zekâ aşamalarından önce Ayarlar'a anahtarı girin.
4. Yapay zekâ kotası dolarsa ertesi gün **Başlat** ile sürdürün; zincir bitince etiketleme sayfasını işaretleyip **Başlat** ile 8b'yi çalıştırın.
5. **Doğrula ve düzelt > 10. Otomatik taşma düzeltme** ile CSS önerisi alın; **Teslim > Teslim paketini oluştur**.

Yeni bir siteyi ilk kez denetlemek

1. **+ Yeni site ekle** ile profil oluşturun.
2. **Hazırlık > Site keşfi**'ni çalıştırın; kayıtta "haritaya N sayfa yazıldı" satırını kontrol edin. Sayı beklenenden çok büyükse hariç desenleri ve sorgu sınırını ayarlayıp keşfi **Taze** başlatın (aynı adım kaldığı yerden devam eder; baştan başlamak için `ciktilar/<profil>/kesif/` klasörünü silin).
3. **Başlat** ile deneme testine, ardından **Kalanları da sırayla çalıştır** ile kalan aşamalara geçin.

Yalnızca taşma düzeltme önerisi almak

Herhangi bir site için profil oluşturup doğrudan **Doğrula ve düzelt > 10. Otomatik taşma düzeltme**'yi çalıştırmak yeterlidir. Site haritası yoksa araç sayfaları sitemap.xml'den ya da linkleri izleyerek kendisi bulur. Çıktı `oneri.css` ve önce/sonra görüntüleridir.

Geliştirici bir düzeltme yaptıktan sonra yeniden kontrol

- Kırık sayfalar için: **Hatalı sayfaları yeniden test et** ve **Kırık sayfa doğrulama**.
- Görsel bulgular için: **7. Bulguları canlı sitede doğrula** (eski bulgular yeni ölçümle karşılaştırılır).
- Tamamını yeniden ölçmek için: Ayarlar'da **Tam testi sıfırdan başlat**'ı açıp tüm site testini çalıştırın.

6. Adımlar

Bu bölüm programdaki bütün adımları sayfa sayfa listeler. Çıktı yolları `ciktilar/<profil>/` klasörüne görelidir.

6.1 Hazırlık

Kurulum, ortam ve sağlayıcı kontrolleri ile sitenin keşfi.

Adım	Ne yapar	Çıktı	Süre
Bağımlılıkları kur	Tüm araçların Python paketlerini ve Playwright Chromium tarayıcısını kurar. İlk kullanımda bir kez çalıştırın.	— (kayıt)	~3 dk
Ortam kontrolü	Bu bilgisayarda Python paketleri, Chromium, Go ve Docker kurulu mu, tek tek gösterir.	— (kayıt)	saniyeler
Yapay zekâ sağlayıcılarını dene	Girilen anahtar ve adreslerle Anthropic, Gemini, OpenAI uyumlu servis ve Ollama'ya bağlanır, kullanılabilir modelleri listeler ve ayarlardaki model adını doğrular.	— (kayıt)	saniyeler
Site keşfi (link haritası)	Profildeki adresten başlayıp gerçek <a href>/form linklerini izler, URL tahmin etmez. Kurallar (hariç desenler, sorgu sınırı, tarayıcı modu) Ayarlar > Site profili'nde. Kesilirse tekrar çalıştırın, kaldığı yerden devam eder.	kesif/site_haritasi.json	dakikalar-saatler
Keşif haritasını kullan	Keşfedilen haritayı bu profilin bundan sonraki tüm adımlarının kaynağı yapar (yeni profillerde zaten öyledir).	— (profil ayarı)	

6.2 Web testi

Sayfalar gerçek bir Chromium tarayıcıda açılır; her sayfa için HTTP durumu, yüklenme süresi, güvenlik başlıkları, Türkçe karakter, erişilebilirlik, mobil uyum, JavaScript hataları ve formlar ölçülür, masaüstü + mobil ekran görüntüsü alınır.

Adım	Ne yapar	Çıktı	Süre
Deneme testi	İlk N sayfayı gerçek tarayıcıda test eder (HTTP, güvenlik başlıkları, Türkçe karakter, erişilebilirlik, mobil, JS hataları, ekran görüntüsü kanıtı).	deneme_output/ report.html, report.xlsx, screenshots/	~2 dk
Tüm site kanıtlı testi	Haritadaki tüm sayfaları parça parça test eder (tara.bat'ın karşılığı). Kesilirse tekrar çalıştırın: yapılmış sayfalar atlanır.	test_output/report.html, report.xlsx, summary.txt, results.jsonl, screenshots/	~2 sa, ~8 GB
Hatalı sayfaları yeniden test et	Yalnızca hata/zaman aşımı alan sayfaları düşük eşzamanlılıkla tekrar dener.	test_output/ (güncellenir)	
Test raporunu yeniden üret	Test çalıştırmadan results.jsonl'den report.html/xlsx/summary üretir.	test_output/ report.html, .xlsx, summary.txt	saniyeler

6.3 Denetimler

Belge erişilebilirliği, bütün bağlantıların sağlığı ve şablon temsilcilerinde görsel/UI kusurları.

Adım	Ne yapar	Çıktı	Süre
Belge denetimi (PDF/Word)	Belge linklerinin erişilebilirliğini ve sızdırılmış yerel yolları denetler.	denetim/ belge_raporu.html, belge_denetimi_excel.csv, documents_audit.csv	
Kırık link denetimi	Sayfalardaki tüm iç ve dış bağlantıları kontrol eder.	linkdenetim/ kirik_link_raporu.html, kirik_linkler_excel.csv, broken_links.csv	saatler
Şablon kümeleme	URL'leri düzen imzasına göre şablonlara indirir (ÖİDB: 17 bin -> ~600), her şablondan temsilci seçer. Görsel denetim ve yerel model bu temsilcilerle çalışır.	template_clusters.json, template_representatives.json (site haritasının yanında)	saniyeler
Görsel/UI kusur denetimi	Şablon temsilcilerinde taşma, kırık görsel, kırpılan metin ve boş render arar; kusuru kırmızı kutulu ekran görüntüsüyle kanıtlar.	gorsel/gorsel_rapor.html, gorsel_denetim.csv	

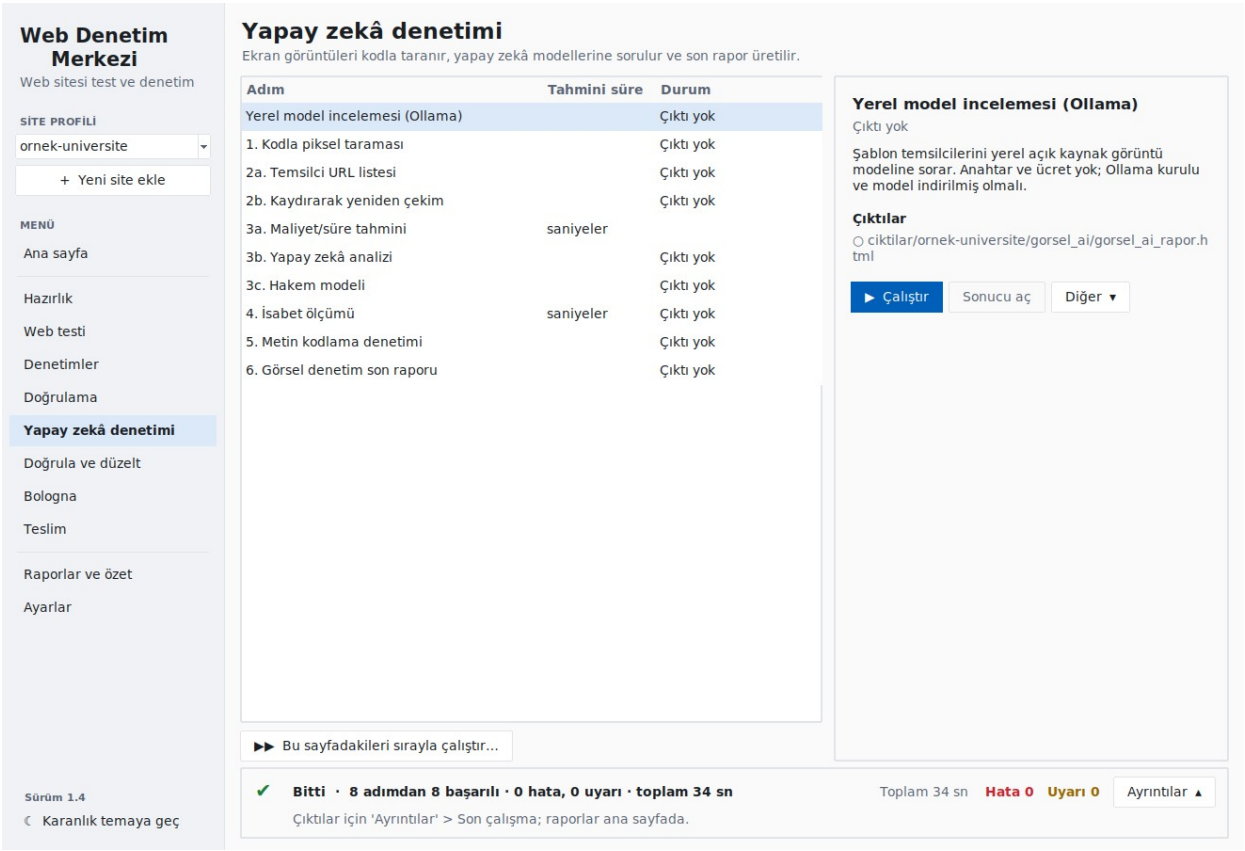
6.4 Doğrulama

Kırık görünen sayfaların gerçekten kırık olduğunun iki bağımsız yolla teyidi.

Adım	Ne yapar	Çıktı	Süre
Kırık sayfa doğrulama (doğrudan erişim)	401/403/404 dönen URL'leri tarayıcıda doğrudan açar: içerik gerçekten gelmiyor mu?	routes_verification.csv	
Buton tıklama doğrulama	Kırık link denetiminin bulduğu site içi kırık adreslere link veren sayfaları açar ve butona gerçekten tıklar: kırık sayfaya mı götürüyor? Önce 'Kırık link denetimi' gerekir.	clicks_verification.csv	

6.5 Yapay zekâ denetimi

Ekran görüntüleri kodla taranır, yapay zekâ modellerine dilim dilim sorulur, modeller birbirini ve piksel ölçümünü doğrular, son rapor üretilir.



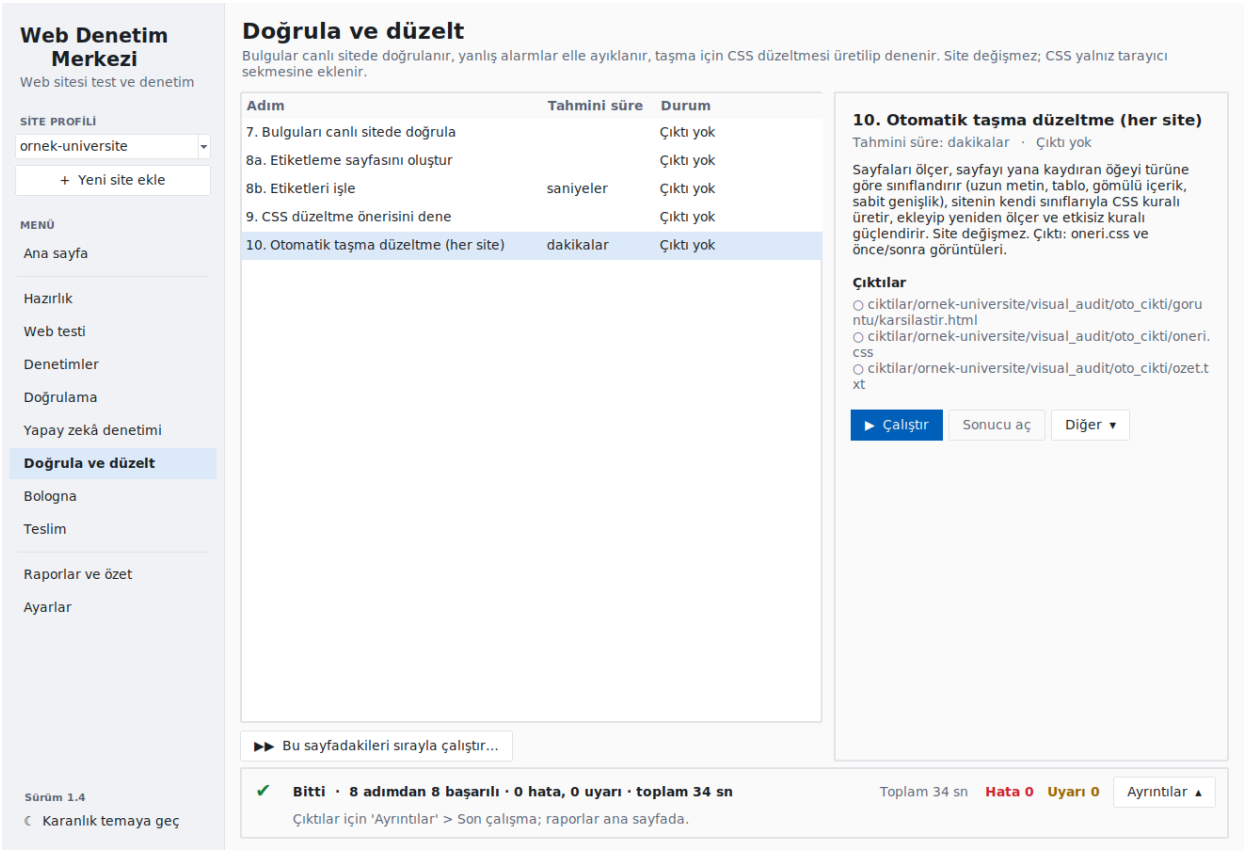
Şekil 11. *Yapay zekâ denetimi sayfası*

Adım	Ne yapar	Çıktı	Süre
Yerel model incelemesi (Ollama)	Şablon temsilcilerini yerel açık kaynak görüntü modeline sorar. Anahtar ve ücret yok; Ollama kurulu ve model indirilmiş olmalı.	gorsel_ai/ gorsel_ai_rapor.html, gorsel_ai.csv	
1. Kodla piksel taraması	Tüm ekran görüntülerini ölçer (taşma, boş sayfa, eksik eş, aynı görünüm) ve AI'ya gidecek temsilcileri seçer.	visual_audit/ denetim_cikti/tarama.csv, ornekler.csv, ozet.txt	
2a. Temsilci URL listesi	Seçilen temsilci görüntülerin URL'lerini site haritasından bulur.	visual_audit/ temsilci_urls.json	
2b. Kaydırarak yeniden çekim	Temsilci sayfaları sona kadar kaydırarak (lazy görseller, sayaçlar) yeniden çeker.	yeniden_cekim/ (screenshots, http_hatali.csv)	
3a. Maliyet/süre tahmini	API çağırmadan seçili modellerle kaç istek ve ne kadar maliyet çıkacağını gösterir.	— (kayıt)	saniyeler
3b. Yapay zekâ analizi	İlk model aşaması. Görüntüleri dilimleyip seçili modellere aynı talimat ve JSON şemasıyla sorar; yapılamayanları aynı çalıştırmada 2 tur daha dener. Sonda kapsamı yazar (ör. 4084 dilimden 3990 tamam). Hiç denenmemiş ya da kota yüzünden kalan dilim varsa adım başarısız biter ve hakeme geçilmez: tekrar çalıştırın, yalnız eksikler denenir.	denetim_cikti/ai_v2/ bulgular.csv, oylama.csv, ozet.txt	

Adım	Ne yapar	Çıktı	Süre
3c. Hakem modeli	Hakem aşaması. Önce ilk modelin her dilimi denediğini kontrol eder; eksik varsa başlamaz. Modellerin anlaşamadığı veya yüksek önem verdiği dilimleri ve ilk modelin denemelere rağmen bakamadığı dilimleri (Gemma 500, zaman aşımı) hakem modele sorar. 3b yeniden çalıştırıldıysa bu adımı ve sonrasını da yeniden çalıştırın.	denetim_cikti/ai_v2/ (güncellenir)	
4. İsabet ölçümü	Hata türü başına hakem onay oranı ve taşmada piksel ölçümüyle precision/recall.	denetim_cikti/ai_v2/ isabet.txt, isabet.csv	saniyeler
5. Metin kodlama denetimi	Görünen metinde Türkçe karakter kaybı (Atat?rk), HTML kodu (') ve mojibake arar.	visual_audit/ metin_kontrol.csv	
6. Görsel denetim son raporu	Kod, metin ve yapay zekâ bulgularını ekran görüntüsü kesitli tek HTML raporda toplar.	denetim_cikti/rapor/ rapor.html	

6.6 Doğrula ve düzelt

Bulguların canlı sitede doğrulanması, yanlış alarmların elle ayıklanması ve taşma için CSS düzeltmesinin üretilip denenmesi.



Şekil 12. *Doğrula ve düzelt sayfası*

Adım	Ne yapar	Çıktı	Süre
7. Bulguları canlı sitede doğrula	Kod, metin ve yapay zekâ bulgusu olan sayfaları yeniden açıp DOM üzerinde ölçer: hata hâlâ var mı, çekimden mi kaynaklanmış? Taşmalarda hatayı üreten öğeyi ve kaç sayfayı etkilediğini çıkarır (kok_neden.csv). Önce yapay zekâ hattı çalışmış olmalı.	visual_audit/dogrulama_cikti/ozet.txt, sonuc.csv, kok_neden.csv	
8a. Etiketleme sayfasını oluştur	Kodla ölçülemeyen ve yanlış alarm sayılan bulgularla doğrulananlardan bir örneklemi tarayıcıda açılan bir sayfada toplar (1 Gerçek, 2 Yanlış, 3 Emin değilim). Bitince sayfadaki 'CSV indir' ile etiketler.csv kaydedilir.	visual_audit/etiket/etiketle.html	
8b. Etiketleri işle	8a'dan sonra, etiketleme sayfasında işaretleyip 'CSV indir' ile kaydettiğiniz etiketler.csv'den tür başına elle ölçülmüş isabeti ve otomatik doğrulamanın isabetini hesaplar. 8a ile art arda değil, etiketlemeden sonra tek başına çalıştırın; dosya yoksa ya da 8a'dan eskiyse bekler. Dosya yolu Ayarlar'dan değiştirilebilir (varsayılan: İndirilenler).	visual_audit/etiket/etiket_ozet.txt, sonuc_etiketli.csv	saniyeler
9. CSS düzeltme önerisini dene	Önerilen CSS'i canlı sayfalara (yalnız tarayıcı sekmesine) ekleyip yeniden ölçer: kaç sayfa düzeldi, hangileri kaldı, yeni bozulan var mı. CSS: Ayarlar'da verilen ya da otomatik düzeltmenin önerisi. Önce 7. adım çalışmış olmalı.	visual_audit/css_cikti/css_ozet.txt, css_sonuc.csv	
10. Otomatik taşma düzeltme (her site)	Sayfaları ölçer, sayfayı yana kaydıran öğeyi türüne göre sınıflandırır (uzun metin, tablo, gömülü içerik, sabit genişlik), sitenin kendi sınıflarıyla CSS kuralı üretir, ekleyip yeniden ölçer ve etkisiz kuralı güçlendirir. Site değişmez. Çıktı: oneri.css ve önce/sonra görüntüleri.	visual_audit/oto_cikti/oneri.css, ozet.txt, goruntu/karsilastir.html	dakikalar

6.7 Bologna

OİBS Bologna Bilgi Paketi'ni tarayıp ilişkileri koruyarak PostgreSQL'e yazar (bkz. Ek A).

Adım	Ne yapar	Çıktı	Süre
PostgreSQL'i başlat (Docker)	Bologna veritabanını Docker ile arka planda başlatır. Kendi PostgreSQL'iniz varsa atlayın ve Ayarlar'daki bağlantı adresini girin.	— (Docker)	
Birim testleri	Go birim testlerini çalıştırır (parser, istemci, CLI). Go kurulu olmalı.	— (kayıt)	
Programları listele	Seçili seviyedeki birim ve programları listeler (veritabanına yazmaz).	— (kayıt)	saniyeler
Tek program tara	Ayarlardaki program numarasını (curSunit) tarar, örn. 1371 Bilgisayar Müh. (MEÜ). Başka bir sitede numarayı 'Programları listele' gösterir.	PostgreSQL	~1 dk

Adım	Ne yapar	Çıktı	Süre
Seviye tara	Seçili seviyedeki (önlisans, lisans, yüksek lisans, doktora) tüm programları tarar.	PostgreSQL	
Tüm siteyi tara	Bologna Bilgi Paketi'nin tamamını PostgreSQL'e yazar.	PostgreSQL	~3,5 sa
Kesilen taramayı sürdür	Tüm site taramasını son 24 saatte yazılmış dersleri atlayarak sürdürür.	PostgreSQL	
Veritabanı istatistikleri	Tablolardaki kayıt sayılarını gösterir.	— (kayıt)	saniyeler

6.8 Teslim

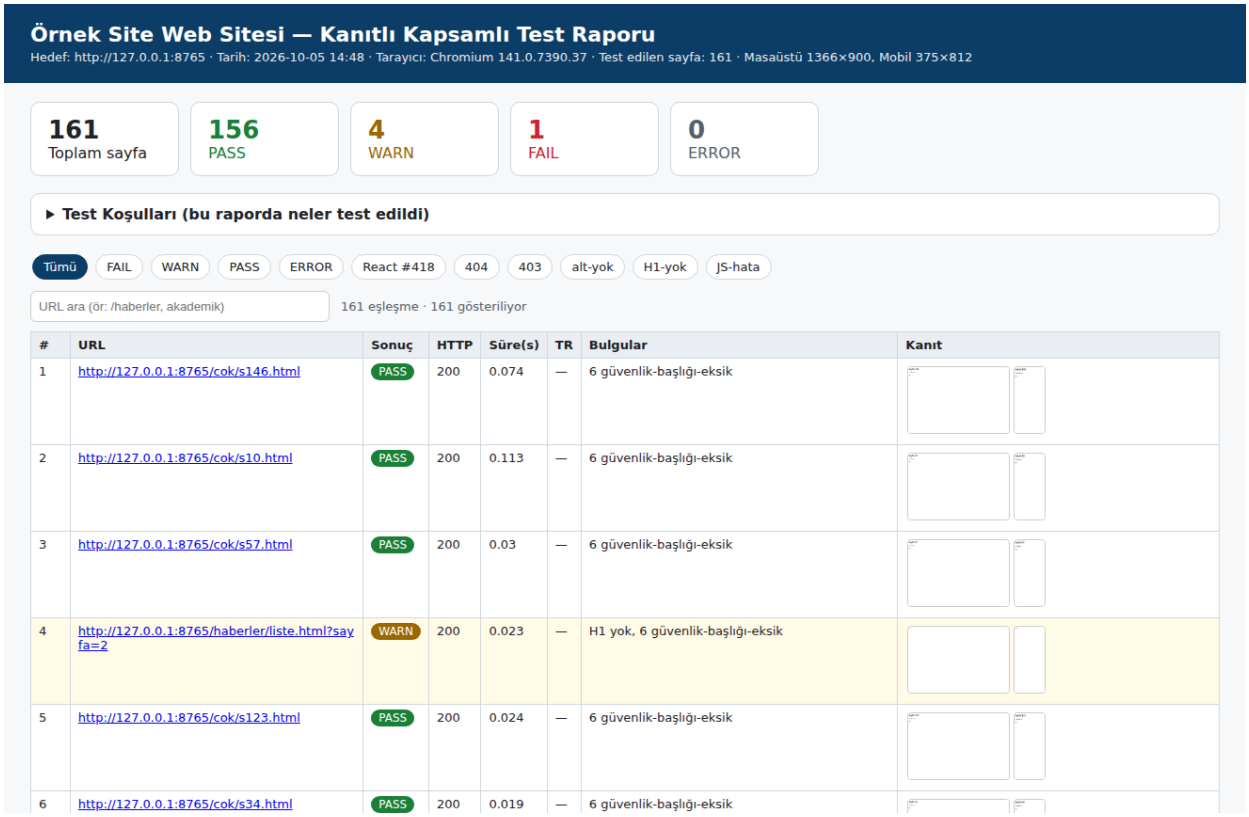
Bütün çıktıları tek klasörde toplar.

Adım	Ne yapar	Çıktı	Süre
Teslim paketini oluştur	Tüm rapor ve verileri anlaşılır adlarla TESLIM klasöründe toplar ve bir içindekiler sayfası (index.html) üretir (toparla.bat'ın karşılığı).	TESLIM/index.html, raporlar/, veri/	saniyeler

7. Sonuçları okumak

7.1 Kanıtlı test raporu (report.html)

`test_output/report.html` tarayıcıda açılır; filtrelenebilir ve sayfalanabilir bir tablodur. Her satır bir sayfadır; küçük resimlere tıklayınca masaüstü ve mobil ekran görüntüsü açılır. Aynı veriler `report.xlsx` (Excel), `summary.txt` (özet) ve `results.jsonl` (ham veri) dosyalarındadır.



Şekil 13. *Kanıtlı test raporu: özet kutucukları, filtreler ve sayfa başına sonuç*

Genel sonuç	Ne zaman verilir
PASS (geçti)	Sayfa 200 döndü ve aşağıdaki uyarılardan hiçbiri yok.
WARN (uyarı)	Sayfa 200 döndü ama şunlardan en az biri var: alt metni olmayan görsel, H1 eksikliği, bozuk Türkçe karakter şüphesi, JavaScript hatası, görsel kusur.
FAIL (kaldı)	Sayfa 200 dışında bir durum döndü (ör. 404, 403, 500) ya da test sırasında sayfa yanıt verdikten sonra hata oluştu.
ERROR (hata)	Sayfaya hiç ulaşılamadı (zaman aşımı, bağlantı hatası). Bu sayfalar **Hatalı sayfaları yeniden test et** ile tekrar denenir.

Raporda ayrıca ölçülen her başlık için sütunlar vardır: HTTP durum ve süre, güvenlik başlıkları (HSTS, CSP, X-Frame-Options, X-Content-Type-Options, X-XSS-Protection, Referrer-Policy), Türkçe karakter, erişilebilirlik, mobil uyum (viewport etiketi, 375 px'te görünüm), konsol/JS hataları (React hydration hataları ayrı işaretlenir) ve formlar.

7.2 Belge ve bağlantı denetimi

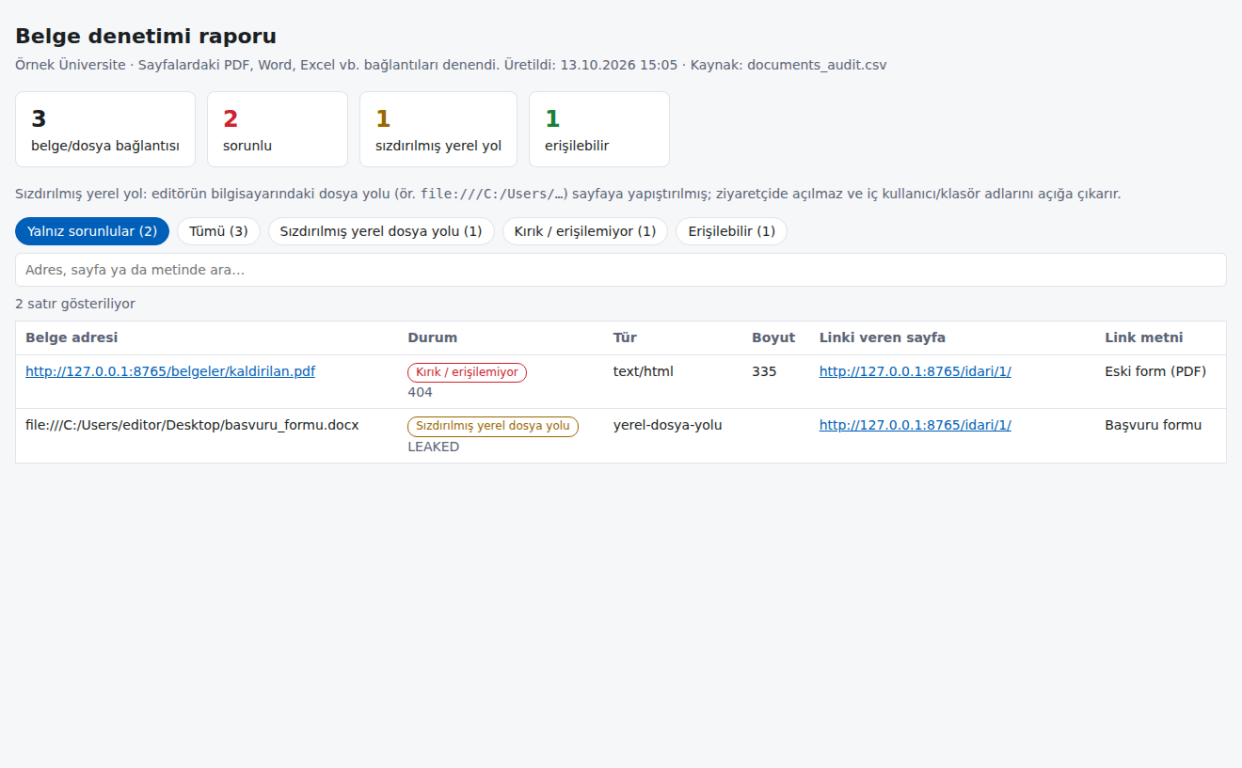
Sonuçları görmenin en kolay yolu **Raporlar ve özet** (ya da ana sayfadaki Raporlar kartı) altındaki **Kırık belge raporu** ve **Kırık link raporu** düğmeleridir. Bunlar süzülebilir, aranabilir HTML raporlar açar:

belge raporu varsayılan olarak yalnız sorunlu belgeleri, link raporu kırık bağlantıları sorun türüne (404, 403, DNS, zaman aşımı…) ve site içi/dış ayrımına göre gösterir; en çok kırık bağlantı olan alan adları da listelenir. Rapor, adımın sonunda kendiliğinden üretilir; eski bir çalıştırmanın CSV'si varsa düğmeye basınca üretilir.

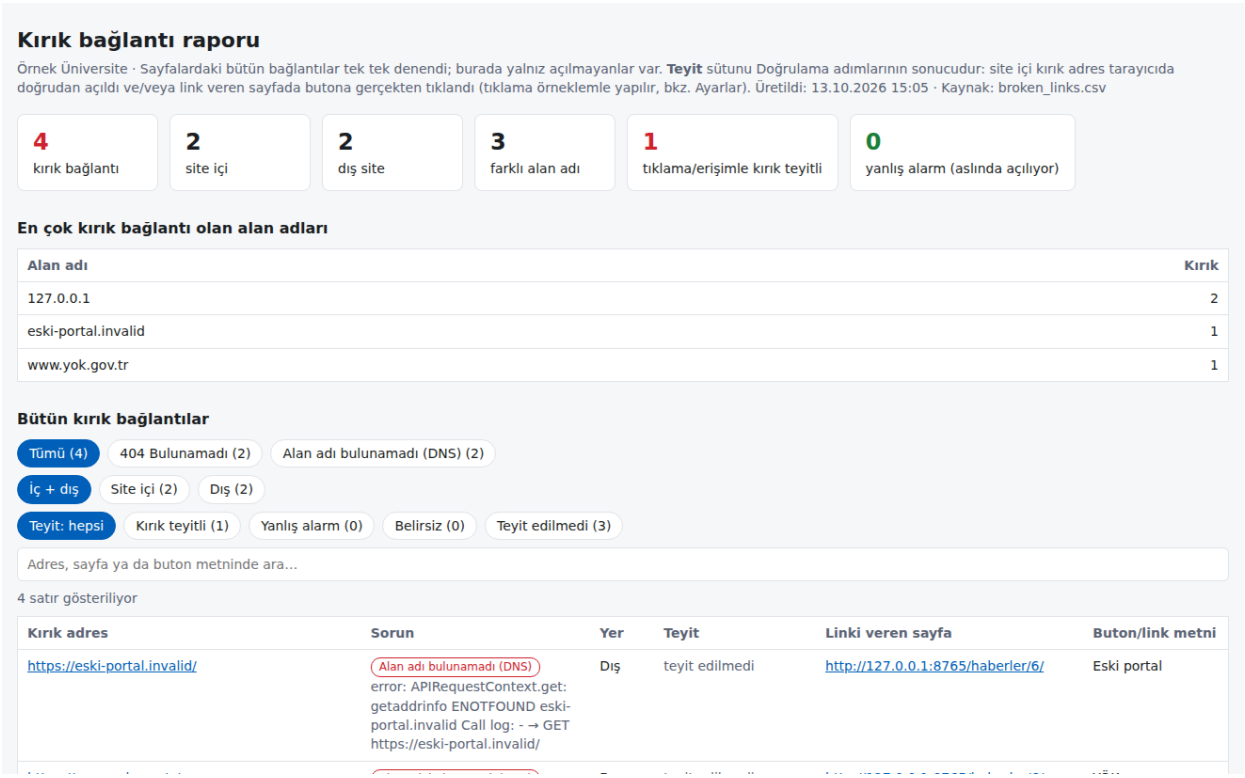
Excel ile çalışmak için aynı klasördeki `belge_denetimi_excel.csv` ve `kirik_linkler_excel.csv` dosyalarını açın: Türkçe Excel'de sütunlar ve karakterler doğru görünür. (Ham `documents_audit.csv` / `broken_links.csv` virgüllü olduğu için Türkçe Excel'de tek sütunda açılır; bunlar araçlar içindir.)

`denetim/documents_audit.csv` her belge bağlantısı için bir satırdır; `flag` sütunu:

flag	Anlamı
erisilebilir	Belge açılıyor.
KIRIK/ERISILEMIYOR	Belge 4xx/5xx dönüyor.
ULASILAMADI	Sunucuya ulaşılamadı (zaman aşımı vb.).
SIZDIRILMIS YEREL DOSYA YOLU	Bağlantı bir bilgisayarın yerel diskini gösteriyor (ör. `C:\Users\...`): hem kırık hem bilgi sızıntısı.



Şekil 14. *Kırık belge raporu: varsayılan olarak yalnız sorunlu belgeler*

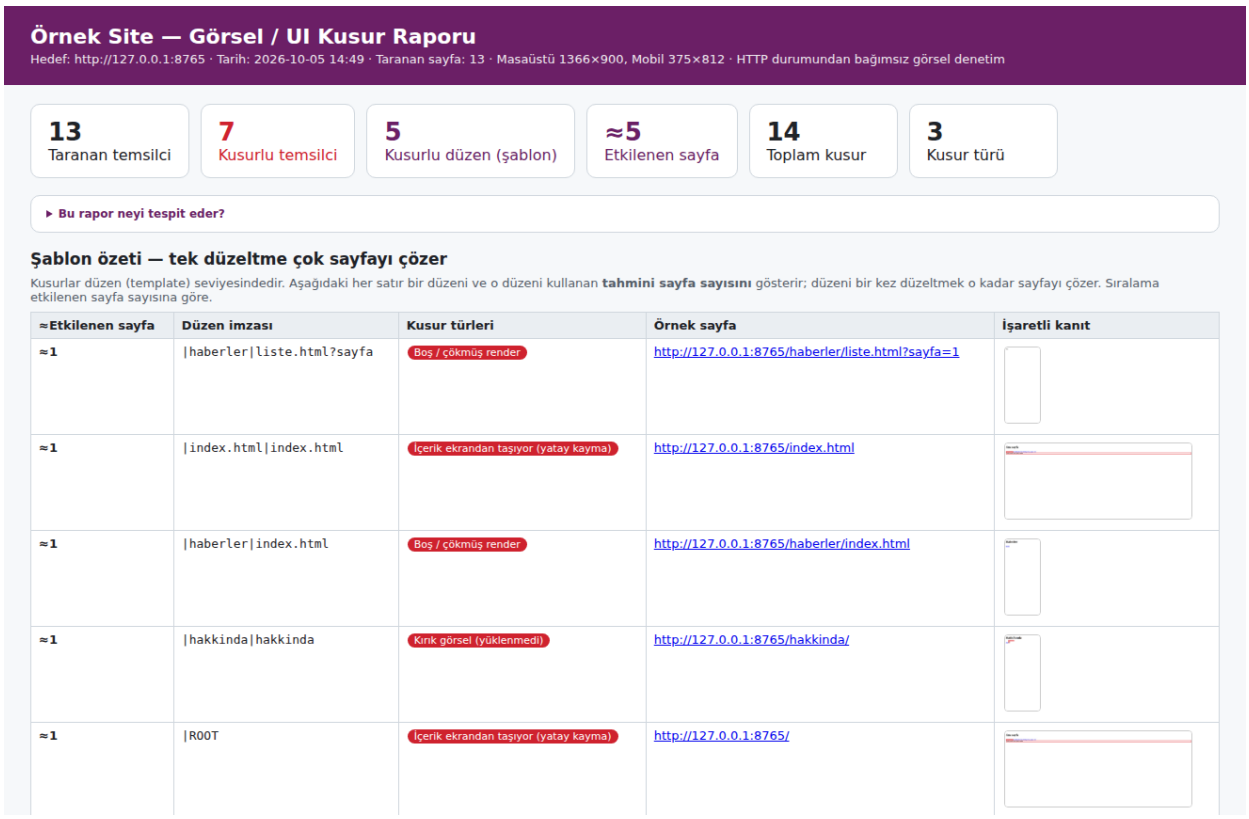


Şekil 15. *Kırık link raporu: sorun türü, site içi/dış ve en çok kırık alan adları*

`linkdenetim/broken_links.csv` kırık bağlantıları, bulunduğu sayfayı (`kaynak_sayfa`) ve butonun metnini (`buton_metni`) verir; düzeltme için hangi sayfada hangi linkin değişmesi gerektiği buradan okunur.

7.3 Görsel/UI kusur raporu

`gorsel/gorsel_rapor.html` şablon özetlidir: bir kusur "bu düzen ≈N sayfayı etkiliyor" diye bir kez raporlanır, kusurlu bölge ekran görüntüsünde kırmızı kutuyla işaretlenir.



Şekil 16. *Görsel/UI kusur raporu: şablon özeti ve işaretli kanıt*

Kusur türü	Anlamı	Öncelik
İçerik ekrandan taşıyor	Ekranda fiilen çizilen, metin/görsel içeren bir öğe görünür alanı aşıyor; kullanıcı sayfayı yana kaydırmak zorunda kalıyor.	Yüksek
Kırık görsel	Görsel yüklenmedi.	Yüksek
Boş / çökmüş render	Sayfa boş ya da içeriksiz çizildi.	Yüksek
Metin kırpılıyor	Metin kutusuna sığmıyor ve kesiliyor.	Orta

7.4 Kırık sayfa doğrulaması

Dosya ve karar	Anlamı
routes_verification.csv: GERÇEKTEN KIRIK	Sayfa doğrudan açıldığında içerik gelmiyor; kırık olduğu kesin.
… İÇERİK GELİYOR (durum X ama tarayıcıda açılıyor)	Sunucu hata kodu döndürse de uygulama sayfayı çiziyor; düşük öncelik.
… ÇALIŞIYOR (200)	Sayfa artık çalışıyor (düzeltilmiş).
clicks_verification.csv: KIRIGA GÖTÜRÜYOR	Sitedeki butona gerçekten tıklandığında kırık sayfaya gidiliyor.
… ÇALIŞAN YERE GİDİYOR	Buton çalışan bir sayfaya gidiyor.
… LİNK/BUTON YOK, REFERRER AÇILMADI	Linki içeren sayfa açılamadı ya da link artık yok.

7.5 Yapay zekâ görsel denetimi

- **Piksel taraması** (`tarama.csv`, `ozet.txt`): yatay taşma, boş sayfa, anormal boy, eksik masaüstü/mobil eşi ve aynı görünen sayfalar kodla kesin olarak ölçülür.
- **Bulgular** (`ai_v2/bulgular.csv`) ve **oylama** (`ai_v2/oylama.csv`): iki model aynı dilimde aynı türü bulduysa **KESİN**, tek model bulduysa **İNCELE**.
- **İsabet** (`ai_v2/isabet.txt`): hata türü başına hakem modelin onay oranı; taşmada piksel ölçümüne göre isabet ve yakalama. İlk çalıştırmada bozuk karakterde %82–92, üst üste binmede %57–60 uyum ölçüldü; taşmada yapay zekâ yerine kod ölçümü esas alınır.
- **Metin kodlama** (`metin_kontrol.csv`): görünen metinde `?` ile kaybolan Türkçe karakter (Atat?rk), ekrana basılan HTML kodu (`'`), mojibake (Ã¼), `&#65533;` ve HTML etiketi.
- **Son rapor** (`denetim_cikti/rapor/rapor.html`): bütün bunları ekran görüntüsü kesitleriyle tek sayfada toplar.

7.6 Canlı doğrulama (7. aşama)

`dogrulama_cikti/ozet.txt` her bulgu türü için kararların sayısını, `sonuc.csv` bulgu başına kararı verir:

Karar	Anlamı
DOGRULANDI	Aynı hata, aynı cihazda (yapay zekâ bulgusunda aynı dikey bölgede) yeniden ölçüldü. Taşmada ayrıca sayfanın kendisi yana kayıyor.
GORULMEDI	Sayfa açıldı ama hata ölçülmedi: düzeltilmiş ya da çekimden kaynaklanmış. Taşmada öğe taşıyor ama sayfa yana kaymıyorsa (kendi kutusunda kayan tablo vb.) yanlış alarm sayılır.
ELLE	DOM ile ölçülemeyen tür (üst üste binme, eksik veri …); işaretli görüntüye bakılmalı → etiketleme.
ACILAMADI	Sayfa yeniden açılamadı.

Kök neden (`kok_neden.csv`): aynı CSS seçicisinden kaynaklanan taşmalar sayfa sayısıyla listelenir. Üstteki satırlar "tek düzeltme, çok sayfa" demektir; geliştiriciye önce bunlar iletilmelidir.

7.7 Elle etiketleme (8. aşama)

1. **8a. Etiketleme sayfasını oluştur** adımından sonra kartta **Çıktıyı aç** ile `etiketle.html` tarayıcıda açılır.
2. Her bulgu için eski çekim ile sayfanın şimdiki hali yan yana gösterilir; ekran kenarı kırmızı çizgiyle işaretlidir.
3. Klavyeyle **1** Gerçek, **2** Yanlış, **3** Emin değilim; ← → ile gezilir. Etiketler tarayıcıda saklanır, sayfa kapanıp açılsa da kaybolmaz.
4. Bitince **CSV indir** ile `etiketler.csv` kaydedilir (varsayılan: İndirilenler klasörü; başka yere kaydettiyseniz Ayarlar'da yolunu verin).
5. **8b. Etiketleri işle** tür başına elle ölçülmüş isabeti (`etiket_ozet.txt`) ve her bulguya insan etiketini (`sonuc_etiketli.csv`) üretir.
6. 8a ile 8b'yi art arda çalıştırmayın: 8b, etiketleme sayfası oluşturulduktan **sonra** indirilmiş bir etiket dosyası bulamazsa "8b bekliyor" deyip durur. Tarayıcı dosyayı `etiketler (1).csv` gibi bir adla kaydettiyse en yenisi kendiliğinden kullanılır.

7.8 CSS düzeltmesi (9. ve 10. aşamalar)

- **10. Otomatik taşma düzeltme**: sayfayı yana kaydıran öğeyi türüne göre sınıflandırır (uzun metin, tablo, gömülü içerik, sabit genişlikli kutu), sitenin kendi sınıflarıyla CSS kuralı yazar, ekleyip yeniden ölçer, etkisizse güçlendirir. `ozet.txt` tur başına kayan/düzelen/yeni bozulan sayısını, `oneri.css` kuralları (her kuralın üstünde kaç sayfayı düzelttiği), `goruntu/karsilastir.html` önce/sonra görüntülerini verir.
- **9. CSS düzeltme önerisini dene**: bir CSS dosyasını (varsayılan: otomatik düzeltmenin önerisi) canlı doğrulamada yana kayan sayfalara ekleyip ölçer. `css_ozet.txt`: düzelen, kalan ve **yeni bozulan** sayfa sayısı.

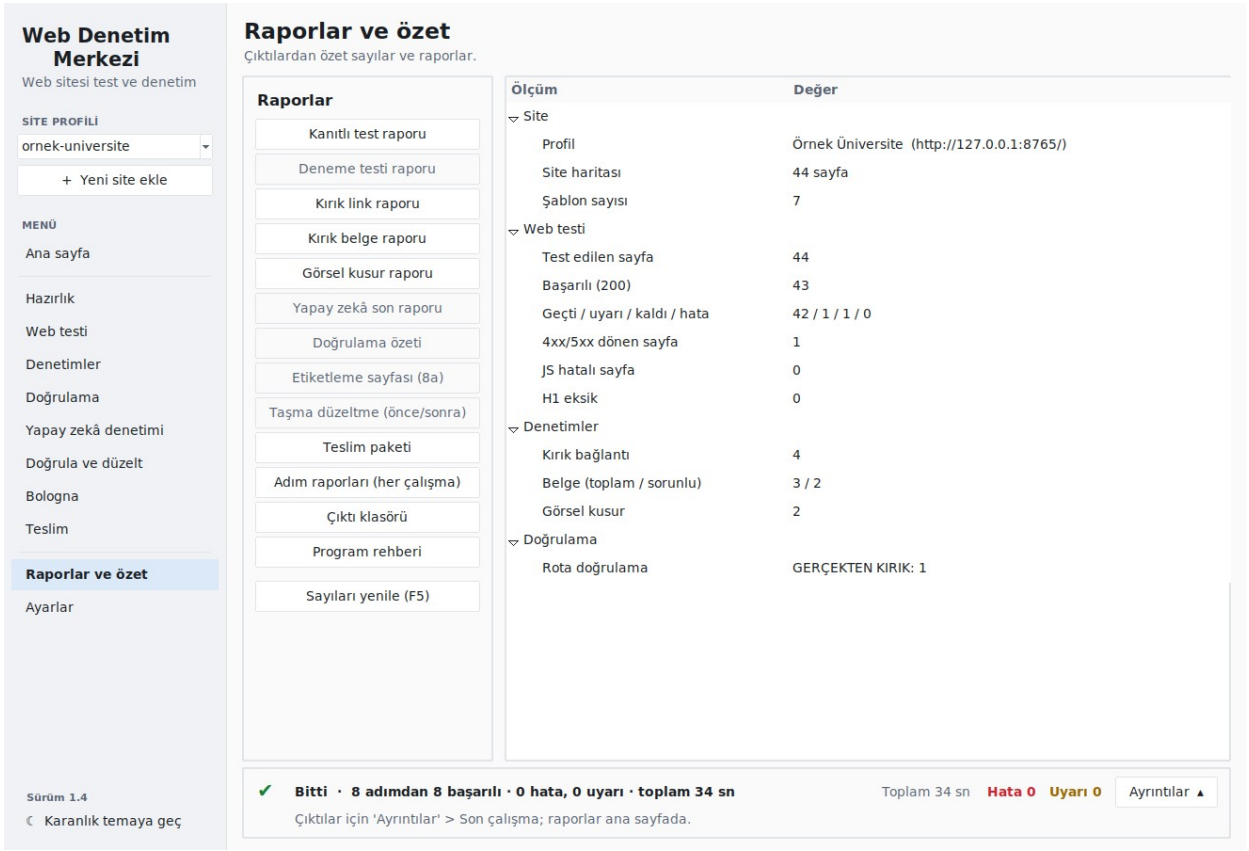
> **Dikkat**
> Ölçüm yalnızca yana kaymayı görür. `oneri.css` geliştiriciye verilmeden önce `karsilastir.html`'deki önce/sonra görüntülerine ve seçicilere bir kez gözle bakılmalıdır. "Yeni bozulan" sıfırdan büyükse o sayfalar elle kontrol edilmelidir.

7.9 Raporlar ve özet sayfası

Menüdeki **Raporlar ve özet** sayfası bütün çıktılardan çıkarılan sayıları tek tabloda gösterir (F5 ile yenilenir) ve raporları açan düğmeleri içerir.

Düğmeler (ana sayfadaki Raporlar kartında da aynıları vardır): Kanıtlı test raporu, Deneme testi raporu, Kırık link raporu, Kırık belge raporu, Görsel kusur raporu, Yapay zekâ son raporu, Doğrulama özeti, **Etiketleme sayfası (8a)**, Taşma düzeltme (önce/sonra), Teslim paketi, Adım raporları ve Çıktı klasörü. Henüz üretilmemiş raporun düğmesi soluk görünür.

> **Not**
> **Etiketleme sayfası** ile **Taşma düzeltme (önce/sonra)** farklı şeylerdir: ilki bulguları 1/2/3 ile işaretlediğiniz sayfadır (8a), ikincisi otomatik CSS önerisinin önce/sonra görüntüleridir (10. adım) ve yalnız bakmak içindir.

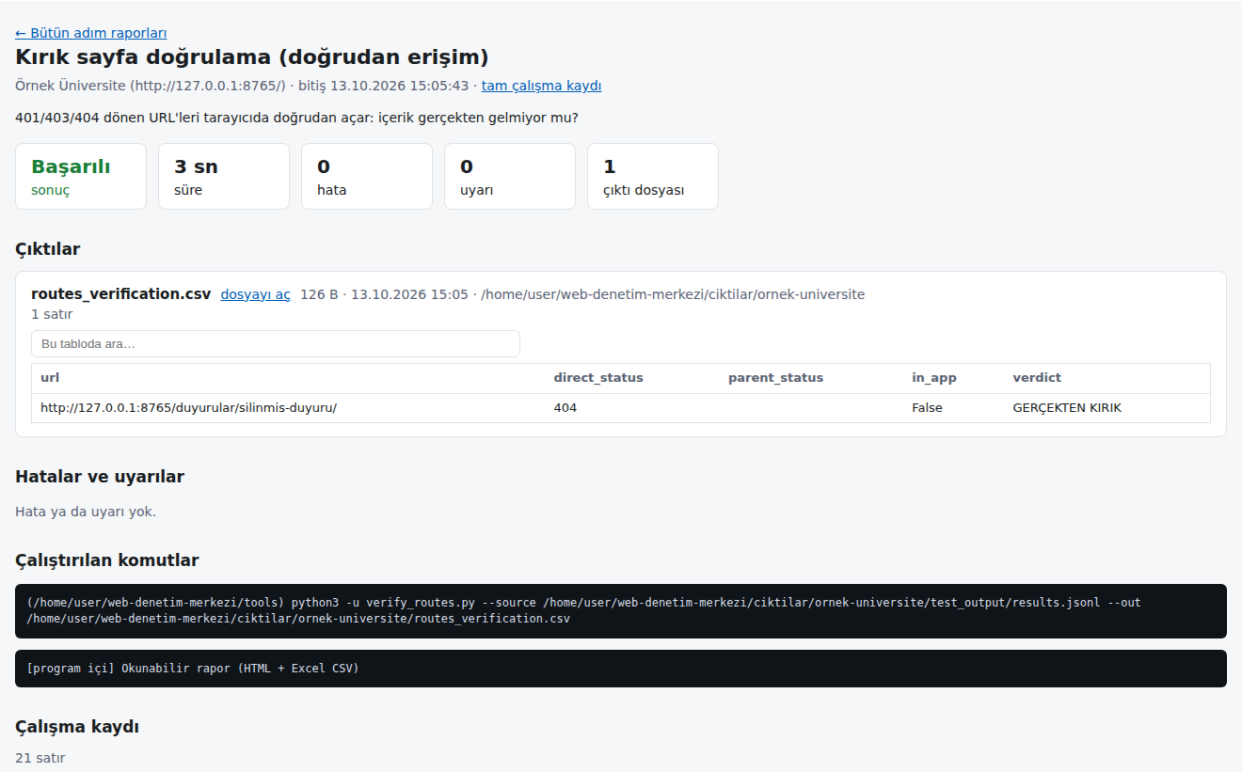


Şekil 17. *Raporlar ve özet sayfası*

7.10 Adım raporları: her adımın her çalışması

Program, hangi adım olursa olsun, adım bittiği anda o adım için bir **adım raporu** yazar; kendi raporu olmayan ara adımlar (doğrulama, kümeleme, piksel tarama, yapay zekâ analizi ve hakem, isabet, metin denetimi, canlı doğrulama, CSS denemesi, Bologna, kurulum ve ortam kontrolü) dahil hiçbir adımın sonucu yalnızca bir CSV'de ya da kayıtta kalmaz. Raporda:

- sonuç (başarılı / başarısız / durduruldu), süre, hata ve uyarı sayıları;
- adımın ürettiği **her dosya**: kendi HTML raporu varsa bağlantısı, CSV'ler aranabilir tablo olarak (ilk 500 satır), TXT'ler metin olarak, JSON'lar özet olarak (alanlar ve öğe sayıları);
- hatalar ve uyarılar listesi, çalıştırılan komutlar ve çalışma kaydının son 400 satırı (tamamına bağlantı).



Şekil 18. *Adım raporu örneği: Kırık sayfa doğrulama — sonuç, CSV çıktısı tablo olarak, hatalar ve komutlar*

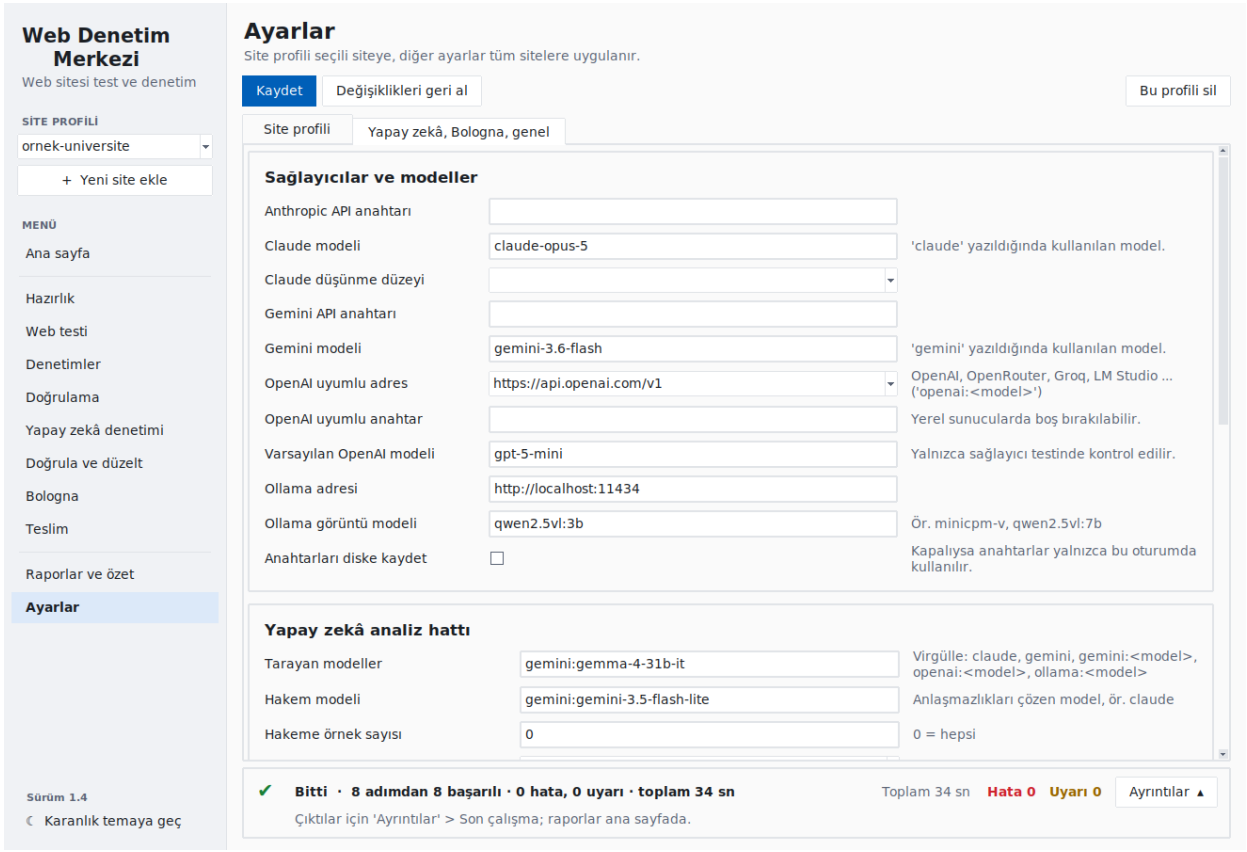
Raporlar `<çıktı klasörü>/adim_raporlari/` altında tarihli adlarla birikir; **Raporlar ve özet → Adım raporları (her çalışma)** hepsini en yeniden eskiye listeleyen dizini açar. Bir adımın en son raporu, adım sayfasındaki ayrıntı kartında **Son adım raporunu aç** ile de açılır. Kırık link raporu ayrıca doğrulama adımlarının teyitlerini (doğrudan erişim, buton tıklama) her kırık bağlantının yanında gösterir.

7.10 Teslim paketi

`TESLIM/` klasöründe `raporlar/` (HTML raporlar, CSS önerisi, son görsel denetim raporu, önce/sonra görüntüleri) ve `veri/` (bütün CSV/XLSX/JSON çıktılar) bulunur; `index.html` bunların açıklamalı listesidir. Ekran görüntüleri büyük olduğu için pakete kopyalanmaz; gerekirse `test_output/screenshots/` ayrıca ZIP'lenir.

8. Yapay zekâ ayarları

8.1 Sağlayıcılar ve model yazımı



Şekil 19. *Ayarlar: yapay zekâ sağlayıcıları ve modeller*

Ayarlar > **Yapay zekâ, Bologna, genel** sekmesinde her sağlayıcının anahtarı, adresi ve varsayılan modeli girilir. **Tarayan modeller** (virgülle birden fazla) ve **Hakem modeli** alanları şu yazımları kabul eder:

Yazım	Sağlayıcı	Gerekenler
`claude` veya `claude:<model>`	Anthropic	Anthropic API anahtarı
`gemini` veya `gemini:<model>`	Google Gemini / Gemma	Gemini API anahtarı (ücretsiz katman yeterli)
`openai:<model>`	OpenAI uyumlu her servis: OpenAI, OpenRouter, Groq, Mistral, Together, LM Studio, vLLM, Ollama'nın OpenAI ucu	Adres (listeden seçilir) ve gerekirse anahtar
`ollama:<model>`	Yerel Ollama	Ollama kurulu ve model indirilmiş olmalı (`ollama pull <model>`)

Hazırlık > Yapay zekâ sağlayıcılarını dene her sağlayıcıya bağlanır, kullanılabilir modelleri listeler ve ayarlardaki model adının listede olup olmadığını söyler. Yeni bir servis için kod değişikliği gerekmez.

8.2 Varsayılan modeller

Görev	Model	Not
Ana tarama	`gemini:gemma-4-31b-it`	Görsel denetimde fiilen kullanılan model; ücretsiz katmanda çalışır.
Hakem	`gemini:gemini-3.5-flash-lite`	Tür bazlı örneklemle (`tur`) modelin isabetini ölçer.
Yerel model	`qwen2.5vl:3b`	Ollama incelemesi. Küçük modeller hataları kaçırabilir; GPU varsa `qwen2.5vl:7b` önerilir.

8.3 Anahtarlar ve güvenlik

- Anahtarlar varsayılan olarak **diske yazılmaz**; yalnızca o oturumda kullanılır. Kalıcı olsun isterseniz **Anahtarları diske kaydet**'i işaretleyin; anahtarlar `ayarlar.json`'a yazılır ve bu dosya git'e girmez.
- Anahtarlar araçlara ortam değişkeni olarak geçirilir ve çalışma kaydına hiçbir zaman açık yazılmaz.
- Kurum içi bir sitenin ekran görüntüleri bulut modellere gönderilir. Bu kabul edilemiyorsa yerel Ollama ya da kurum içi OpenAI uyumlu bir sunucu kullanın.

8.4 Kota, hız ve maliyet

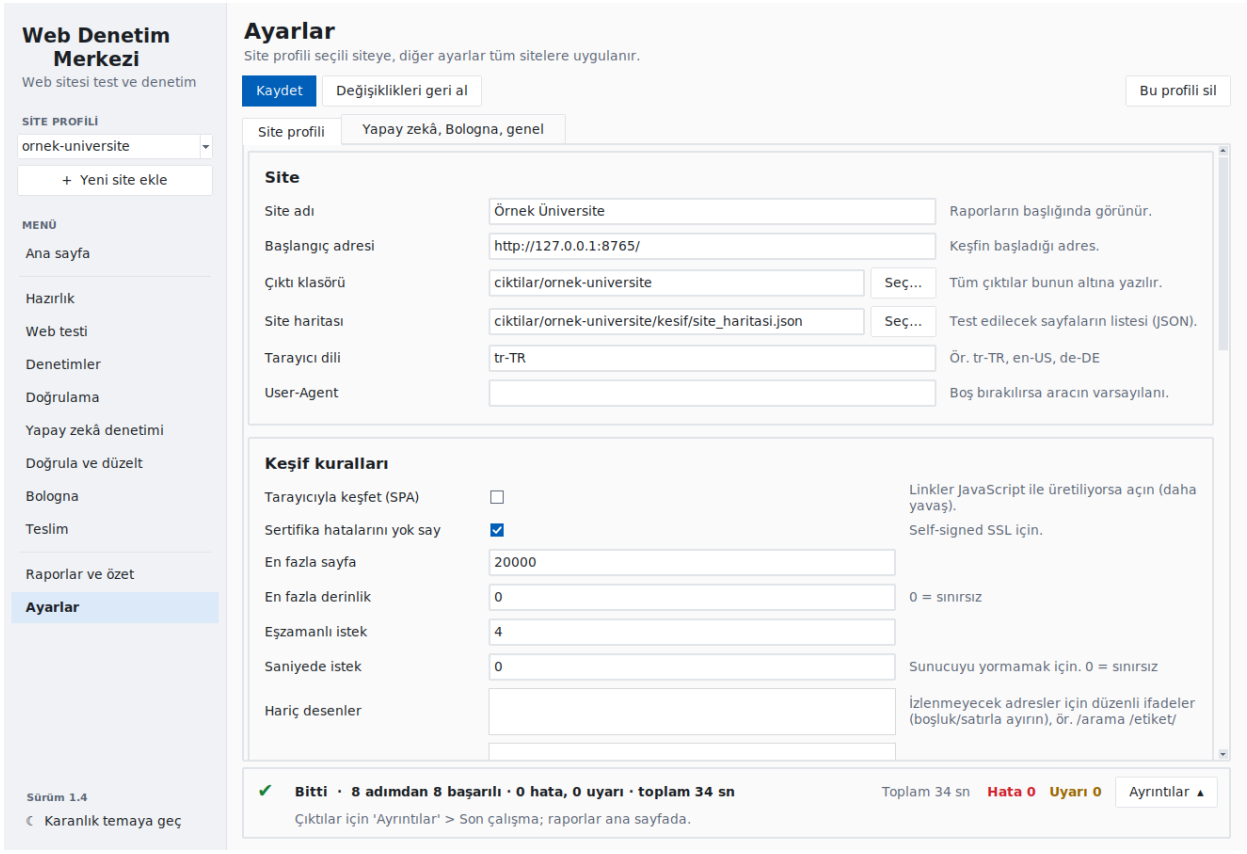
- **Dakikada istek sınırı** (varsayılan 8): ücretsiz Gemini katmanının dakikalık sınırına takılmamak için. Ücretli hesapta 0 (sınırsız) yapılabilir.
- Günlük kota dolarsa analiz o model için durur; ertesi gün aynı adım kaldığı yerden devam eder.
- **Gemma "gemma_500_kalici" hatası:** Gemma bazı görüntülerde sunucu hatası (500) verir. Araç görüntüyü önce PNG, sonra JPEG olarak dener; ikisi de düşerse dilimi "bakılamadı" işaretler. Bu dilimler aynı çalıştırmada 2 tur daha denenir; yine düşenlere **3c. Hakem modeli** bakar. İsterseniz 3c'den önce **3b**'yi bir kez daha çalıştırıp ilk modele bir şans daha verebilirsiniz.
- **3a. Maliyet/süre tahmini** API çağırmadan istek sayısını ve Claude için tahmini maliyeti gösterir. Önce **Sayfa sınırı (deneme)** ile küçük bir tur yapmanız önerilir.
- Claude kullanıyorsanız **Claude toplu istek (Batch)** maliyeti yarıya indirir; sonuç genelde bir saat içinde gelir.

8.5 Modele giden talimat

Her ekran görüntüsü cihaza göre dilimlenir (masaüstü 1024×1024, mobil 375×1334 piksel, görüntü başına en fazla 3 dilim) ve her dilim modele ayrı sorulur. Modele, sitenin tanımını ve dilini içeren İngilizce bir talimat, dilimin cihaz ve konum bilgisi, görüntü ve cevabın uyması gereken JSON şeması gider. Model yalnızca gerçek bir ziyaretçinin fark edeceği görsel hataları, her biri için görünen metni kanıt olarak kopyalayarak raporlar; emin olmadığı şeyi yazmaz. Hata türleri: yatay taşma, üst üste binme, kesik metin, kırık görsel, hizasızlık, düşük kontrast, bozuk karakter, açıklanamayan boşluk, bozuk gezinme, eksik veri, diğer.

Talimattaki site tanımı ve dil **Ayarlar > Site profili > Yapay zekâ talimatı**'ndan değiştirilir. Talimatın tam metni `tools/visual_audit/analiz.py` dosyasındadır.

9. Ayarlar başvurusu



Şekil 20. *Ayarlar: site profili*

Site profili ayarları seçili siteye, **Yapay zekâ, Bologna, genel** ayarları bütün sitelere uygulanır. Varsayılan sütunu yeni bir profilin değerini gösterir; ÖİDB profilinde bazı değerler farklıdır.

9.1 Site profili

Site

Ayar	Açıklama	Varsayılan
Site adı	Raporların başlığında görünür.	profile göre
Başlangıç adresi	Keşfin başladığı adres.	profile göre
Çıktı klasörü	Tüm çıktılar bunun altına yazılır.	profile göre
Site haritası	Test edilecek sayfaların listesi (JSON).	profile göre
Tarayıcı dili	Ör. tr-TR, en-US, de-DE	tr-TR
User-Agent	Boş bırakılırsa aracın varsayılanı.	(boş)

Keşif kuralları

Ayar	Açıklama	Varsayılan
Tarayıcıyla keşfet (SPA)	Linkler JavaScript ile üretiliyorsa açın (daha yavaş).	kapalı

Ayar	Açıklama	Varsayılan
Sertifika hatalarını yok say	Self-signed SSL için.	açık
En fazla sayfa	—	20000
En fazla derinlik	0 = sınırsız	0
Eşzamanlı istek	—	4
Saniyede istek	Sunucuyu yormamak için. 0 = sınırsız	5
Hariç desenler	İzlenmeyecek adresler için düzenli ifadeler (boşluk/satırla ayırın), ör. /arama /etiket/	(boş)
Yalnızca bu desenler	Boş = tüm site.	(boş)
Silinecek sorgu parametreleri	Sonsuz kombinasyon üreten parametreler (sort, filtre, oturum ...).	utm_source utm_medium utm_campaign fbclid gclid sessionid
Yol başına sorgu varyasyonu	0 = sınırsız	50
sitemap.xml'i de kullan	—	açık
robots.txt'ye uy	—	kapalı
Alt alan adlarını dahil et	—	kapalı

Test

Ayar	Açıklama	Varsayılan
Eşzamanlı sekme	4 önerilir; yüksek değer sunucuyu zorlar.	4
Sayfa zaman aşımı (sn)	—	30
Deneme testi sayfa sayısı	—	50
Tam testte parça boyu	—	2000
Tam testi sıfırdan başlat	Kapalıysa yapılmış sayfalar atlanır (kaldığı yerden devam).	kapalı
Tablet görüntüsü de al	—	kapalı
Tarayıcıyı görünür çalıştır	Hata ayıklama için.	kapalı
Şablon başına temsilci	—	3
Rota doğrulama sınırı	0 = hepsi	0
Tıklama doğrulama sınırı	—	40

Görsel doğrulama ve düzeltme

Ayar	Açıklama	Varsayılan
Otomatik düzeltme: sayfa sayısı	Ölçülecek en fazla sayfa (şablonlara yayılarak seçilir).	200
Otomatik düzeltme: cihaz	—	mobile
Otomatik düzeltme: tur sayısı	En fazla düzeltme turu.	4
Önce/sonra görüntüsü	Kaç sayfanın görüntüsü alınsın (0 = alma).	12
Denenecek CSS dosyası	Boş: otomatik düzeltmenin önerisi; o da yoksa hazır oneri.css.	(boş)
Etiketleme örneklemi	Doğrulananlardan rastgele kaç bulgu.	30
İndirilen etiketler.csv	Boş: İndirilenler klasörü.	(boş)

Yapay zekâ talimatı

Ayar	Açıklama	Varsayılan
Site tanımı (İngilizce)	Modele 'You are a UI QA reviewer for ...' diye verilir.	a website
Sitenin dili (İngilizce)	Ör. Turkish, English, German	Turkish

Bologna (OİBS)

Ayar	Açıklama	Varsayılan
OİBS Bologna adresi	OİBS kullanan herhangi bir üniversitenin Bologna adresi; alan adı yeter (ör. obs.ornek.edu.tr). Her sitenin verisi veritabanında kendi şemasına yazılır (Mersin Üni.: public).	https://obs.mersin.edu.tr/oibs/bologna/
Program no (curSunit)	—	1371
Seviye	—	lisans

9.2 Yapay zekâ, Bologna, genel

Sağlayıcılar ve modeller

Ayar	Açıklama	Varsayılan
Anthropic API anahtarı	—	(boş)
Claude modeli	'claude' yazıldığında kullanılan model.	claude-opus-5
Claude düşünme düzeyi	—	(boş)
Gemini API anahtarı	—	(boş)
Gemini modeli	'gemini' yazıldığında kullanılan model.	gemini-3.6-flash
OpenAI uyumlu adres	OpenAI, OpenRouter, Groq, LM Studio ... ('openai:<model>')	https://api.openai.com/v1

Ayar	Açıklama	Varsayılan
OpenAI uyumlu anahtar	Yerel sunucularda boş bırakılabilir.	(boş)
Varsayılan OpenAI modeli	Yalnızca sağlayıcı testinde kontrol edilir.	gpt-5-mini
Ollama adresi	—	http://localhost:11434
Ollama görüntü modeli	Ör. minicpm-v, qwen2.5vl:7b	qwen2.5vl:3b
Anahtarları diske kaydet	Kapalıysa anahtarlar yalnızca bu oturumda kullanılır.	kapalı

Yapay zekâ analiz hattı

Ayar	Açıklama	Varsayılan
Tarayan modeller	Virgülle: claude, gemini, gemini:<model>, openai:<model>, ollama:<model>	gemini:gemma-4-31b-it
Hakem modeli	Anlaşmazlıkları çözen model, ör. claude	gemini:gemini-3.5-flash-lite
Hakeme örnek sayısı	0 = hepsi	0
Hakem örnek grubu	—	tur
Claude toplu istek (Batch, %50 ucuz)	—	kapalı
Sayfa sınırı (deneme)	0 = hepsi	0
Dakikada istek sınırı	Ücretsiz kotalar için, 0 = sınırsız	8
Eşzamanlı istek	—	4
İsabet: ana model	sonuclar.jsonl'deki model adı	gemini:gemma-4-31b-it
İsabet: hakem model	—	gemini:gemini-3.5-flash-lite

Bologna veritabanı

Ayar	Açıklama	Varsayılan
Çalıştırma biçimi	docker: Go/PostgreSQL kurmadan	docker
PostgreSQL adresi	Yerel modda kullanılır.	postgres://bologna:bologna@localhost:5432/bologna?sslmode=disable
Havuz ders detayları	--pool-details	açık
Plan yılı sayısı	1, 3 ya da all	1
Paralel program	—	2
Saniyede istek	2'nin üstü önerilmez.	2

Genel

Ayar	Açıklama	Varsayılan
Python yorumlayıcısı	Araçları çalıştıracak Python.	profile göre
Arayüz görünümü	sade: hızlı (önerilen). modern: Windows 11 görünümü, ama Windows'ta sayfa geçişleri ve tema değişimi belirgin yavaştır. Programı yeniden açınca geçerli olur.	sade
Hata olursa sonraki adımlarla sürdür	Kapalıyken birden çok adımlı bir çalışmada bir adım başarısız olursa sonrakiler çalışmaz.	kapalı
Ayrıntılı kayıt (hata ayıklama)	Her komutun klasörü, ortamı ve argümanları kayda yazılır; Bologna -v ile çalışır.	kapalı

10. Komut satırı

Bütün adımlar arayüz açmadan da çalıştırılabilir (sunucuda, zamanlanmış görevde). Kurulumdan sonra Windows'ta `.venv\Scripts\python`, Linux/macOS'ta `./baslat.sh` kullanın:

```
python web_denetim.py --liste                  # bütün adımlar
python web_denetim.py --goster tam_test        # çalışacak komutlar (çalıştırmaz)
python web_denetim.py --calistir deneme        # bir ya da birkaç adım
python web_denetim.py --calistir tam-hat       # tam hat
python web_denetim.py --calistir ai-hatti      # yapay zekâ hattı
python web_denetim.py --calistir belge link --devam-et   # hata olsa da sürdür
python web_denetim.py --durum                  # çıktılardan özet
python web_denetim.py --profiller              # profiller
python web_denetim.py --yeni-profil "Örnek Üniversite" https://www.ornek.edu.tr/
python web_denetim.py --profil ornek-universite --calistir kesif deneme
```

Adım kimlikleri (`deneme`, `tam_test`, `gd_oto` …) `--liste` çıktısında ve 6. bölümdeki tablolarda adların karşılığıdır. Her araç ayrıca tek başına da çalışır: `python tools/<araç>.py --help`.

11. Klasörler ve dosyalar

```
web-denetim-merkezi/
├── BASLAT.bat, KURULUM.bat        Windows başlatma ve kurulum
├── baslat.sh, kurulum.sh          Linux / macOS
├── web_denetim.py                 Program (arayüz + komut satırı)
├── ayarlar.json                   Genel ayarlar (git'e girmez)
├── profiller/                     Site profilleri
├── data/meu-oidb/                 ÖİDB site haritası ve şablon kümeleri
├── merkez/                        Programın kendisi
├── tools/                         Araçlar (tek başına da çalışır)
│   ├── tester_v3_evidence.py      Kanıtlı test, belge/link/görsel denetim
│   ├── kesif.py                   Site keşfi
│   └── visual_audit/              Görsel denetim (1-10. aşamalar)
├── bologna-scraper/               Bologna (Go)
├── docs/                          Bu kılavuz (Word ve PDF)
└── ciktilar/<profil>/             Bütün çıktılar (git'e girmez)
    ├── kesif/                     site_haritasi.json
    ├── deneme_output/, test_output/   report.html, report.xlsx, screenshots/
    ├── denetim/, linkdenetim/, gorsel/
    ├── routes_verification.csv, clicks_verification.csv
    ├── yeniden_cekim/, dogrulama/, dogrulama_css/, gorsel_ai/
    ├── visual_audit/              denetim_cikti/, dogrulama_cikti/, etiket/,
    │                              css_cikti/, oto_cikti/, metin_kontrol.csv
    ├── merkez_kayitlari/          Her çalışmanın kaydı
    └── TESLIM/                    Teslim paketi (index.html)
```

12. Sorun giderme

Belirti	Olası neden	Çözüm
BASLAT.bat'a basınca pencere bir an açılıp kapanıyor; ya da "Web Denetim Merkezi açılamadı" penceresi çıkıyor	Bir program dosyası eksik, eski ya da bozuk (çoğunlukla dosyalar tek tek indirilerek güncellendiğinde).	Projeyi **ZIP ile** güncelleyin (2.5). Hatanın tamamı program klasöründeki `baslatma_hatasi.txt` dosyasındadır; komut isteminde `.venv\Scripts\python.exe web_denetim.py` ile de görülebilir.
BASLAT.bat açılıp kapanıyor, "Python bulunamadı"	Python kurulu değil ya da PATH'te yok.	Python'u python.org'dan "Add python.exe to PATH" işaretli kurun, BASLAT.bat'ı yeniden çalıştırın.
Kurulum "Paketler kurulamadı" / "Chromium indirilemedi" diyor	İnternet ya da kurum proxy'si engelliyor.	Bağlantıyı ve proxy ayarlarını kontrol edip KURULUM.bat'ı tekrar çalıştırın. Kurum ağında `HTTPS_PROXY` ortam değişkeni gerekebilir.
Arayüz açılmıyor, "tkinter" hatası	Python Tkinter'sız kurulmuş (Linux'ta sık).	Linux: `sudo apt install python3-tk`; Windows: Python'u python.org kurulumuyla yeniden kurun. Komut satırı Tkinter'sız da çalışır.
Ana sayfada "Kurulum eksik" uyarısı	Paket ya da Chromium eksik.	Uyarıdaki **Bağımlılıkları kur** düğmesi; ardından **Ortam kontrolü**.
Arayüz yavaş, sayfa geçişlerinde ya da tema değişiminde öğeler tek tek çiziliyor	Modern görünüm (sv-ttk) seçilmiş. Bu tema her öğeyi saydam resimlerle çizer; Windows ölçümünde sayfa geçişi 5–7 kat, tema değişimi ~5 kat yavaş çıktı.	**Ayarlar** → **Genel** → **Arayüz görünümü**: `sade` (varsayılan) seçip kaydedin ve programı yeniden açın. Renkler ve düzen aynı kalır.
Testte çok sayıda ERROR / zaman aşımı	Sunucu yavaş ya da eşzamanlılık yüksek; VPN kopmuş.	Eşzamanlılığı 2–3'e, zaman aşımını 45–60 sn'ye ayarlayın; **Hatalı sayfaları yeniden test et**. VPN'i kontrol edin.
Bütün sayfalar FAIL, durum boş	Siteye ulaşılamıyor (adres yanlış, VPN yok).	Site adresini tarayıcıda açmayı deneyin; profilin başlangıç adresini kontrol edin.
Keşif bitmiyor, sayfa sayısı çok büyük	Filtre/takvim gibi sonsuz kombinasyonlu adresler.	Hariç desenler, silinecek parametreler ve sorgu sınırını ayarlayın; `kesif/` klasörünü silip yeniden başlatın.
Keşif çok az sayfa buldu	Linkler JavaScript ile üretiliyor (SPA).	**Tarayıcıyla keşfet**'i açın.
Yapay zekâ: "model bu hesapta kullanılamıyor (404)"	Model adı yanlış ya da hesapta yok.	**Yapay zekâ sağlayıcılarını dene** ile kullanılabilir modelleri görün, ayarı düzeltin.
Yapay zekâ: "günlük kota doldu"	Ücretsiz katman kotası.	Ertesi gün aynı adımı çalıştırın; kaldığı yerden devam eder.
Yapay zekâ analizi uzun süre ilerlemiyor gibi görünüyor	Ekrana yalnız hatalar yazılır; Gemma yavaştır.	`[i/N] ✓ … ✗ …` satırındaki sayaç ilerliyorsa çalışıyordur (satır arası 8–15 dk normal). 30 dakikadan uzun süre yeni satır yoksa adımı durdurup yeniden başlatın; tamamlananlar atlanır. Bilgisayarın uykuya geçmediğinden emin olun.

Belirti	Olası neden	Çözüm
Yapay zekâ: 3b "İlk model tamamlanmadı" ile başarısız bitti ya da 3c "İlk model tamamlanmadan hakem başlamaz" diyor	Analiz yarıda kalmış ya da kota dolmuş; bazı dilimler ilk modele hiç sorulmamış.	**3b**'yi tekrar çalıştırın (günlük kota bittiyse ertesi gün): yalnız eksikler denenir. Ana sayfadaki "Yapay zekâ: ilk model" satırı tamamlanınca 3c'yi çalıştırın.
Yapay zekâ: çok sayıda "gemma_500_kalici" ya da sonda "N istek yapılamadı"	Gemma sunucu hatası, kota ya da bağlantı sorunu.	Adım bitince **3b**'yi tekrar çalıştırın: yalnız yapılamayanlar denenir. Sonra **3c, 4, 5, 6**'yı yeniden çalıştırın. Yine düşen Gemma dilimlerine hakem bakar.
Yapay zekâ adımı "ornekler.csv yok" diyor	Önceki adımlar çalışmamış.	Yapay zekâ hattını baştan (1. adımdan) çalıştırın.
Canlı doğrulama "results.jsonl yok" diyor	Doğrulama çekimi yapılmamış ya da yarıda kalmış.	7. adımı yeniden çalıştırın.
"Etiketleri işle" CSV'yi okuyamıyor	Dosya başka yerde ya da sayfadan indirilmemiş.	Etiketleme sayfasındaki **CSV indir**'i kullanın; dosya İndirilenler'de değilse Ayarlar'da yolunu verin.
Bologna: "Program bulunamadı: docker"	Docker Desktop kurulu değil ya da çalışmıyor.	Docker Desktop'ı kurup başlatın; ya da Ayarlar'da Bologna modunu "yerel" yapıp Go + PostgreSQL kurun.
Docker Desktop: "WSL is not installed" / "WSL needs updating"	Docker, Windows'ta WSL 2 üzerinde çalışır.	Yönetici PowerShell'de `wsl --install --no-distribution` (yarım kuruluysa `wsl --update`), sonra bilgisayarı yeniden başlatın. "Sanallaştırma kapalı" derse BIOS/UEFI'de Intel VT-x / AMD SVM'i açın.
Docker Desktop "Starting the Docker Engine..." ekranında kalıyor	İlk açılışta motor 1–5 dk'da başlar.	10 dakikayı geçerse Docker Desktop'ı kapatıp açın (balina simgesi > Quit); olmazsa bilgisayarı yeniden başlatın.
Disk doldu	Ekran görüntüleri.	Eski profillerin `ciktilar/<profil>/test_output/screenshots/` klasörlerini arşivleyip silin.

Çözemediğiniz bir sorunda **Hatalar ve uyarılar > Panoya kopyala** ile listeyi ve ilgili adımın `merkez_kayitlari/` altındaki kayıt dosyasını paylaşın. Ayarlar > Genel > **Ayrıntılı kayıt** açıkken kayıt, her komutun klasörünü ve argümanlarını da içerir.

13. Sık sorulan sorular

Program siteyi değiştirir mi?

Hayır. Sayfalar yalnızca okunur. CSS denemeleri yalnızca programın açtığı tarayıcı sekmesine eklenir; site dosyalarına dokunulmaz.

Sunucuyu zorlar mı?

Eşzamanlı sekme sayısı ve saniyede istek ayarlarıyla sınırlanır. Ön-prodüksiyon sunucularında 4 eşzamanlı sekme önerilir; 10 gibi yüksek değerler zaman aşımlarına yol açabilir.

Yarıda kalan işi baştan mı çalıştırmam gerekir?

Hayır. Aynı adımı yeniden çalıştırın; test, keşif, yapay zekâ analizi ve Bologna kaldığı yerden devam eder.

Yapay zekâ analizinde hata alan istekler ne olur? Adımı tekrar çalıştırırsam hepsini baştan mı yapar?

Hayır, yalnız yapılamayanları (sunucu hatası, kota, bağlantı) yeniden dener; başarılı olanlar atlanır. Analizi yeniden çalıştırdıysanız hakem (3c) ve sonraki adımları da yeniden çalıştırın ki son rapor güncellensin.

Yapay zekâ olmadan kullanılabilir mi?

Evet. Test, belge, link, görsel denetim, doğrulama, canlı doğrulama, otomatik taşma düzeltme ve teslim yapay zekâ gerektirmez.

Bulgulara ne kadar güvenebilirim?

Kodla ölçülen bulgular (HTTP durumu, taşma, kırık link) kesindir. Yapay zekâ bulguları iki modelle ve canlı doğrulamayla süzülür; yine de elle etiketlemeyle bir örneklem kontrol edilmelidir. İsabet tabloları (7.5, 7.7) bunun ölçüsünü verir.

Bir sonucu bir başkasına nasıl iletirim?

Teslim > Teslim paketini oluştur ve `TESLIM/` klasörünü paylaşın; `index.html` her dosyanın ne olduğunu açıklar.

Başka bir bilgisayarda kaldığım yerden devam edebilir miyim?

Evet: program klasörünü (`ciktilar/` dahil) kopyalayın ve orada BASLAT.bat'ı çalıştırın; `.venv` yeniden kurulur.

Ek A. Bologna

Bologna adımları, OİBS kullanan bir üniversitenin **Bologna Bilgi Paketi**'ni tarayıp birimler, programlar, program çıktıları, ders planları ve ders detaylarını (öğrenme çıktısı × program çıktısı katkı matrisi dahil) ilişkileri koruyarak PostgreSQL'e yazar.

1. Docker Desktop'ı kurup başlatın. Windows'ta Docker WSL 2 ister: "WSL is not installed" derse yönetici PowerShell'de `wsl --install --no-distribution` çalıştırıp bilgisayarı yeniden başlatın. Docker Desktop'ta sol altta "Engine running" yazınca hazırdır (ilk açılış 1–5 dk). Docker istemiyorsanız Ayarlar'da modu "yerel" yapıp Go ve PostgreSQL kurun, PostgreSQL adresini girin.
2. **Bologna > PostgreSQL'i başlat (Docker)**.
3. Deneme için **Tek program tara** (varsayılan 1371: Bilgisayar Mühendisliği; ilk seferde araç derlendiği için 3–5 dk, sonra ~1 dk).
4. Tamamı için **Tüm siteyi tara** (~3,5 saat). Yarıda kalırsa **Kesilen taramayı sürdür**.
5. **Veritabanı istatistikleri** ile tablo kayıt sayılarını görün. Veritabanı adresi: `postgres://bologna:bologna@localhost:5432/bologna` (DBeaver, pgAdmin ile bağlanılabilir).

Başka üniversiteler

Varsayılan site Mersin Üniversitesi'dir. Aynı OİBS altyapısını kullanan başka bir üniversite için profilde (Ayarlar > Site profili > **OİBS Bologna adresi**) ya da yeni site eklerken adresi girin; yalnızca alan adı yeter (ör. `obs.ornek.edu.tr`). Sonra **Programları listele** ile program numaralarını görüp **Program no**'ya yazın ve **Tek program tara** ile deneyin.

- **Veriler karışmaz:** her üniversite aynı veritabanında kendi **şemasına** yazılır. Mersin Üniversitesi `public` şemasındadır; diğerlerinin şeması adresten türetilir (`obs.ornek.edu.tr → ornek`). SQL'de `SET search_path TO ornek;` ya da `ornek.courses` biçiminde sorgulanır. **Veritabanı istatistikleri** hangi site ve şemaya baktığını en üstte yazar.
- Bir şemaya başka bir sitenin verisi yazılmak istenirse araç durur ve hata verir.
- Parser Mersin Üniversitesi sayfalarıyla doğrulanmıştır. Sayfalarını özelleştirmiş bir üniversitede bazı alanlar boş gelebilir; önce tek programla deneyin.

Veri modeli, örnek SQL sorguları ve ayrıntılar: `bologna-scraper/README.md`.

Ek B. Geliştirme notları

- Yeni bir araç programa `merkez/adimlar.py`'deki listeye bir adım eklenerek bağlanır; arayüz, komut satırı, durum ve kayıtlar yeni adımı kendiliğinden kullanır.
- Testler: `python -m unittest discover -s tests -v` ve `cd bologna-scraper && go test ./...`. GitHub Actions bunları Windows ve Linux'ta, Python 3.9 ve 3.12 ile çalıştırır.
- **Sözleşme testleri**, programın araçlara gönderdiği her seçeneğin aracın kodunda tanımlı olduğunu denetler; araçlar güncellendiğinde uyumsuzluğu yakalar.
- Görsel denetim `gorsel-denetim` reposunda geliştirilir; `tools/visual_audit/` ve `tools/tester_v3_evidence.py` oradaki `denetim/` ve `cekim/tester.py` ile aynı araçlardır ve üç yönlü birleştirmeyle güncellenir.